AGENTIC MULTI-LLM VALIDATION SYSTEM: LABEL-FREE QUALITY ASSURANCE FOR LOCALLY HOSTED LARGE LANGUAGE MODELS

Submitted in partial fulfillment of the requirements

of the degree of

Bachelor of Technology in Computer Engineering

By

Swaroop Naik (Roll No. 23102A0001)

Nidhi Bhawari (Roll No. 23102A0006)

Devanshi Mahajan (Roll No. 23102A0069)

Under the Guidance of

Dr. Suja Jayachandran

Department of Computer Engineering

(An Autonomous Institute Affiliated to University of Mumbai)

Vidyalankar Institute of Technology

Wadala (E), Mumbai-400437

University of Mumbai

2026-27

CERTIFICATE OF APPROVAL

This is to certify that the project entitled

"Agentic Multi-LLM Validation System: Label-Free Quality Assurance for Locally Hosted Large Language Models"

is a bonafide work of

Swaroop Naik (Roll No. 23102A0001)
Nidhi Bhawari (Roll No. 23102A0006)
Devanshi Mahajan (Roll No. 23102A0069)

submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of Undergraduate in "Computer Engineering".

_____

Guide

(Dr. Suja Jayachandran)

_____

Head of Department

(Ravindra Sangale)

_____

Principal

(Sangeeta Joshi)

Project Report Approval for
B.Tech in Computer Engineering

This project report entitled "Agentic Multi-LLM Validation System: Label-Free Quality Assurance for Locally Hosted Large Language Models" by

Swaroop Naik (23102A0001)
Nidhi Bhawari (23102A0006)
Devanshi Mahajan (23102A0069)

is approved for the degree of Bachelor of Technology in Computer Engineering.

Examiners

1. _____

2. _____

Date: _____

Place: _____

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Sr. No.	Name of Student	Roll No.	Signature
1.	Swaroop Naik	23102A0001	
2.	Nidhi Bhawari	23102A0006	
3.	Devanshi Mahajan	23102A0069	

Date: _____

Acknowledgements

We express our sincere gratitude to our guide, Dr. Suja Jayachandran, Department of Computer Engineering, Vidyalankar Institute of Technology, for the guidance, patience and critical questioning that shaped this work. The insistence on measuring claims rather than asserting them is the single habit that most improved this project, and several results in Chapter 5 exist only because a comfortable-looking number was questioned instead of accepted.

We thank Ravindra Sangale, Head of the Department of Computer Engineering, and Sangeeta Joshi, Principal, for providing the laboratory facilities and the computational resources without which a fully local, six-model pipeline would not have been feasible.

We are grateful to the project coordinators for their guidance on the evaluation schedule and documentation standards, and to our peers for reviewing early demonstrations and asking the awkward questions that exposed more than one hidden assumption.

Finally, we thank our families for their patience during the long unattended experimental runs that this project required.

Abstract

Large Language Models produce fluent text that is frequently wrong, and the standard remedy - asking a second model to grade the first - inherits well documented pathologies of its own. Evaluator models are known to prefer answers in particular prompt positions, to prefer longer answers, and to prefer text they themselves generated. This project asks a narrower and more tractable question: can the quality of a locally hosted question-answering system be validated when no human-annotated ground truth is available at all?

The system presented here answers each question twice, using two independently prompted local models, and then validates the pair rather than trusting either. An independent Verifier and an independent Judge score both candidates blind against an eight-factor, eighty-point rubric. Semantic similarity, cross-model agreement and two external encyclopedia checks are combined with the Judge's total into a single weighted confidence score. Any answer below a calibrated threshold is regenerated automatically, up to a bounded number of attempts, and a sixth agent synthesises the final response. The pipeline runs entirely offline with no external paid interfaces.

Validation is label-free by construction. A self-generated corpus of 5,000 questions across ten domains was produced, and the decision threshold calibrated against censored data using a deliberately injected control group, receiver-operating-characteristic analysis with Youden's J statistic, and a 2,000-sample bootstrap, yielding an adopted operating point of 0.72 with a 95 per cent confidence interval of [0.69, 0.75].

The investigation also produced a negative result of independent interest. Measured across all 5,000 records, with prompt position and evaluator verdict crossed as controls and answer length excluded as a competing explanation, the original synthesising agent preferred its own generated answer by 18.1 percentage points and the first prompt slot by a further 10.8. Replacing it with a model that is neither generator was verified by a controlled replay of 1,600 synthesis calls: the position effect falls from 80.5 to 50.7 per cent, but a smaller length-mediated lean appears in its place rather than parity. Four measurement errors were caught before reaching any reported conclusion, and are documented rather than omitted.

Table of Contents

Acknowledgements 5
Abstract 6
List of Figures 10
List of Tables 11
List of Symbols and Abbreviations 12
Chapter 1 Introduction 13
1.1 Background 13
1.2 Motivation 13
1.3 Problem Statement 14
1.4 Objectives 14
1.5 Scope 15
1.6 Limitations 15
1.7 Contributions 16
1.8 Organisation of the Report 16
Chapter 2 Literature Review 18
2.1 Overlap-Based Automatic Metrics 18
2.2 Embedding-Based Metrics 18
2.3 LLM-as-a-Judge 19
2.4 Documented Biases of Model-Based Evaluators 19
2.5 Multi-Agent Orchestration 20
2.6 Retrieval-Augmented Generation and External Grounding 20
2.7 Threshold Selection and Diagnostic Statistics 21
2.8 Self-Consistency and Ensemble Approaches 21
2.9 Calibration and Uncertainty 22
2.10 Comparative Summary 22
2.11 Identified Gap 23
Chapter 3 System Architecture 24
3.1 Design Principles 24
3.2 Pipeline Overview 24
3.3 Agent Roles and Model Assignment 25
3.4 The Eight-Factor Rubric 26
3.5 Semantic and Agreement Signals 27
3.6 External Fact-Checking 28
3.7 The Weighted Confidence Score 28
3.8 Why Six Signals, and Why These Weights 29
3.9 The Regeneration Gate 30
3.10 Bias Controls 30
3.11 Retrieval Subsystem 31
Chapter 4 Implementation 32
4.1 Technology Stack 32
4.2 Agent Abstraction 32
4.3 Prompt Design 32
4.4 Concurrency and Memory Constraints 33
4.5 Streaming Interface 33

4.6 The Event Protocol 34
4.7 Frontend 35
4.8 Retrieval Store 35
4.9 Data Model 36
4.10 Dataset Generation Harness 36
4.11 Operational Incidents 36
4.12 Resilience 37
Chapter 5 Experiments and Results 39
5.1 Experimental Setup 39
5.2 Corpus Characteristics 39
5.3 Regeneration Behaviour 40
5.4 Does Regeneration Improve the Answer? 41
5.5 Judge Selection: Four Models and a Methodology Error 42
5.6 Calibrating the Rubric: Four Revisions 44
5.7 An Evaluator Coupling Error 46
5.8 Verifier Discrimination 46
5.9 Per-Criterion Score Behaviour 47
5.10 Inter-Evaluator and Cross-Model Agreement 48
5.11 Are the Six Signals Independent? 48
5.12 Availability of the External Signals 49
5.13 Position Skew in the Judge 50
5.14 Comparing the Two Generators 50
5.15 Computational Cost 52
5.16 Threshold Calibration: the Censoring Problem 52
5.17 Threshold Calibration: Three Independent Routes 53
5.18 Threshold Calibration: Choosing the Operating Point 54
5.19 Evaluator Bias: Measuring the Combiner 55
5.20 Evaluator Bias: Decomposition 56
5.21 Evaluator Bias: the Correction and its Limits 57
5.22 Measurement Errors Detected and Corrected 59
5.23 The Retrieval Path Was Inert Throughout 61
5.24 Threats to Validity 62
Chapter 6 System Testing and Validation 64
6.1 Testing Strategy 64
6.2 Score Computation and Renormalisation 64
6.3 Rubric Parsing 64
6.4 Gate and Regeneration Logic 65
6.5 Randomisation Balance 65
6.6 Persistence and Restart Behaviour 66
6.7 Failure Modes and Degradation 66
6.8 End-to-End Verification 67
Chapter 7 Conclusion and Recommendations 68
7.1 Conclusions 68
7.2 Recommendations 68
Chapter 8 Future Scope 70
8.1 Eliminating the Position Effect by Construction 70

8.2 A Human-Annotated Validation Study 70
8.3 Calibrating the Retrieval Threshold 71
8.4 Distillation of the Judge 71
8.5 Broader Signal Set and Adaptive Weighting 71
8.6 Generalisation 72
References 73
Appendices 75
Appendix A: Agent Configuration 75
Appendix B: Evaluator Prompt Structure 75
Appendix C: The Eight-Factor Rubric in Full 75
Appendix D: Confidence Score Computation 76
Appendix E: Database Schema 76
Appendix F: Threshold Calibration Procedure 77
Appendix G: Combiner Bias Measurement Procedure 77
Appendix H: Full ROC Table 78
Appendix I: A Worked Example 79
Appendix J: Verbatim Evaluator Output 80
Appendix K: Glossary of Terms as Used in This Report 81
Appendix L: Development Chronology 82
Appendix M: Reproducing the Results 82

List of Figures

Figure 3.1 Six-stage pipeline architecture, showing the regeneration loop and the signals feeding the confidence score

Figure 4.1 Streaming interface: the pipeline renders stage by stage as it executes

Figure 5.1 Dataset generation monitor at completion of the 5,000-question corpus

Figure 5.2 Distribution of the weighted confidence score, all attempts against final answers, with the adopted threshold marked

Figure 5.3 Attempts per question, and the reason each regeneration was triggered

Figure 5.4 Receiver operating characteristic of the confidence score against observed improvement, sampling-corrected

Figure 5.5 Youden's J across candidate thresholds, showing the near-flat plateau between 0.70 and 0.74

Figure 5.6 Bootstrap distribution of the optimal threshold over 2,000 weighted resamples

Figure 5.7 Combiner preference crossed by prompt position, model identity and judge verdict, with the two effects separated

List of Tables

Table 2.1 Evaluation approaches and their applicability to this project
Table 3.1 Agent roles, models and decoding parameters
Table 3.2 The eight-factor evaluation rubric
Table 3.3 Confidence score signal weights
Table 5.1 Corpus composition
Table 5.2 Threshold operating points
Table 5.3 Combiner bias, effects separated
Table 5.4 Measurement errors and corrections
Table 5.5 Per-criterion scores for the winning answer
Table 5.6 External signal availability across 8,432 attempts
Table 5.7 The two generators compared by true identity
Table 5.8 Controlled replay, 400 questions x 2 models x 2 orders
Table 5.9 Score change across regeneration attempts
Table 5.10 Pearson correlation between confidence signals
Table 5.11 Rubric revisions and their effect on Judge behaviour
Table 5.12 Five candidate judges on one real question-answer pair
Table 5.13 Head-to-head on 20 real triples (N=40 scores per model)
Table 5.14 CompassJudger-1 flat-scoring rate
Table 5.15 Final judge comparison (N=40 blind evaluations each)
Table 6.1 Observed failure modes and system response
Table H.1 Sampling-corrected ROC across all candidate cutoffs
Table I.1 Signals for question 4, attempt 2
Table K.1 Terms as used in this report

List of Symbols and Abbreviations

Symbol / Abbreviation	Expansion
API	Application Programming Interface
AUC	Area Under the Curve
BERT	Bidirectional Encoder Representations from Transformers
BLEU	Bilingual Evaluation Understudy
CI	Confidence Interval
CPU	Central Processing Unit
DDL	Data Definition Language
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
J	Youden's J statistic (sensitivity + specificity − 1)
LLM	Large Language Model
MMR	Maximal Marginal Relevance
NLP	Natural Language Processing
pp	Percentage Points
RAG	Retrieval-Augmented Generation
ROC	Receiver Operating Characteristic
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
SSE	Server-Sent Events
SQL	Structured Query Language
VRAM	Video Random Access Memory
τ (tau)	Confidence threshold below which an answer is regenerated
r	Pearson correlation coefficient
p	Probability value of a statistical test
n	Sample size

Chapter 1 Introduction

1.1 Background

Large Language Models have moved in a very short time from research artefacts to components that ordinary software is expected to depend upon. They summarise documents, answer questions, draft code and power customer-facing assistants. Their central practical weakness is well known and has not been solved: they generate text that is fluent, confident and internally coherent whether or not it is correct.

This failure mode is qualitatively different from the failures engineers are used to. A database that cannot reach its host raises an exception. A parser given malformed input reports a syntax error. A language model given a question it cannot answer does not raise anything at all; it produces a well-formed paragraph in the same register as a correct one, and the burden of detecting the error falls entirely on the reader. A wrong answer is not merely possible, it is indistinguishable in tone from a right one.

The response that has emerged across industry and research is to use a second model as an evaluator, a pattern commonly called LLM-as-a-judge. Its appeal is that it scales. Evaluation becomes a function call rather than a human reading answers, which means it can run on every output rather than on a sample. It is also, however, a measurement instrument built from the same material as the thing being measured, and a growing body of work shows that such instruments carry systematic biases of their own.

A second constraint motivates this work specifically. Much of the published evaluation literature assumes access either to human preference labels or to a frontier commercial model acting as a reference judge. Both assumptions fail for a department laboratory, which has neither an annotation budget nor an API budget, and both fail for any deployment handling data that cannot leave the institution - medical records, student records, internal documents. This project therefore restricts itself to models that run locally, on a single workstation, with no external paid interfaces of any kind.

1.2 Motivation

Two observations motivated the specific direction this project took.

The first is that quality assurance for language model outputs is usually discussed as though a ground truth were available. Benchmarks publish accuracy figures against reference answers; leaderboards rank systems by agreement with human preference. In a deployed

system answering open-ended questions from real users, neither exists. There is no reference answer to compare against, and no annotator standing by. If quality can only be assessed where labels exist, then it cannot be assessed where it actually matters.

The second is that the evaluation machinery is rarely itself evaluated. A pipeline that uses one model to grade another produces numbers, and those numbers are generally reported as measurements. But if the grader systematically prefers answers in a particular prompt position, or prefers text written in its own style, then the numbers are contaminated, and any improvement measured against them may be an artefact of the instrument rather than a change in the system. This is not a hypothetical concern: it is documented in the literature, and as Chapter 5 shows, it was present and substantial in this project's own first design.

The combination suggests a research question that is narrower than "how good is this system" and considerably more tractable: can the quality of a locally hosted question-answering system be validated, and improved, when no human-annotated ground truth is available - and can the validation machinery be held to the same standard of scrutiny it applies to the system it measures?

1.3 Problem Statement

Given a question-answering system assembled from locally hosted Large Language Models, and given no human-annotated ground truth for its outputs, determine whether the quality of its answers can be validated, quantified and improved - and determine the extent to which the validation machinery itself is biased.

The second clause is not decorative. If the evaluation apparatus silently prefers one model, one answer position or one writing style, then every quality figure it produces is contaminated, and improvements measured against it may be artefacts. A validation system that cannot be turned on itself is of limited scientific value.

Three sub-problems follow. First, a quality signal must be constructed from evidence that is available without labels. Second, any decision threshold applied to that signal must be chosen from data rather than intuition - and the obvious way to do so fails, because a threshold that gates a feedback loop censors exactly the observations needed to calibrate it. Third, the apparatus must be measured for position and identity bias with the confounds properly controlled, which requires an experimental design rather than an inspection.

1.4 Objectives

- Design and implement a multi-agent question-answering pipeline in which generation and evaluation are performed by different models, running entirely offline on local hardware.
- Define a multi-signal confidence score that combines model-internal and external evidence, so that no single signal dominates the quality decision.
- Establish an automatic regeneration mechanism gated on that confidence score, with a bounded number of attempts and a guarantee that regeneration cannot degrade the shipped answer.
- Calibrate the regeneration threshold from data rather than intuition, accounting for the fact that the available observations are censored by the gate itself.
- Generate a corpus large enough for the calibration to be statistically meaningful, without human annotation, and resilient enough to survive a multi-day unattended run.
- Measure the validation apparatus for position bias and self-preference bias, with answer length controlled as a competing explanation, and correct what is found.
- Document measurement errors made during the project, including those caught before they influenced any conclusion.

1.5 Scope

The system is scoped to open-ended factual and technical question answering in English, over ten general knowledge domains: Science and Technology, History and Geography, Health and Medicine, Economics and Finance, Philosophy and Ethics, Environment and Climate, Psychology and Behaviour, Society and Culture, Mathematics and Logic, and Space and Astronomy.

It is not scoped to mathematical reasoning requiring symbolic manipulation, to code generation, to multi-turn dialogue, or to languages other than English. An optional retrieval mode accepts user-supplied documents, but document-grounded question answering is a secondary capability rather than the object of study.

1.6 Limitations

Three limitations are stated at the outset rather than deferred to the conclusion, because they bound what the results in Chapter 5 can be taken to mean.

First, the validation is label-free. This means it can establish internal consistency, evaluator discrimination, relative improvement under regeneration, and the calibration of an

operating threshold. It cannot establish absolute factual accuracy. A claim of the form "the system is 86 per cent correct" is not supported by this methodology and is not made anywhere in this report.

Second, the tuned values are specific to this model set. The threshold, the signal weights and the choice of judge were derived against these six models on this corpus. The protocol is expected to transfer; the numbers are not. That expectation has not been tested and is stated as a hypothesis in Chapter 8.

Third, the position effect documented in Section 5.8 is mitigated but not eliminated. Randomising presentation order makes the effect unbiased with respect to model identity, and changing the synthesising model removes the identity component, but a generative synthesiser still over-weights whatever it reads first. This is recorded as an open problem rather than presented as solved.

1.7 Contributions

- A label-free validation methodology for a multi-model question-answering pipeline, requiring neither human annotation nor a commercial reference judge.
- A technique for calibrating a decision threshold under censoring, by deliberately injecting a control group of observations the gate would otherwise never produce.
- A threshold derived by three independent statistical routes - a two-proportion significance test, ROC analysis with Youden's J, and a 2,000-sample bootstrap - which converge on one narrow band.
- A decomposition of synthesiser bias into position and identity components, with answer length tested and excluded as a competing explanation, which overturned the project's own prior architectural decision.
- A documented account of four measurement errors detected during the work, including the reasoning that exposed each, offered as a contribution rather than concealed.

1.8 Organisation of the Report

Chapter 2 surveys the literature on automated evaluation of generated text, on the documented biases of model-based evaluators, and on the statistical methods used later for threshold selection, and identifies the gap this work addresses. Chapter 3 presents the system architecture: the six stages, the rubric, the confidence score and the bias controls,

with the reasoning behind each design decision. Chapter 4 describes the implementation, including the data generation harness and the resilience mechanisms that an unattended multi-day experimental run required. Chapter 5 presents the experiments and results, and is the substantive chapter: corpus characteristics, judge selection, verifier discrimination, threshold calibration, the combiner bias investigation, and the measurement errors detected along the way. Chapter 6 reports the system testing and validation that establishes the implementation behaves as specified, which is a different kind of evidence from the experiments of Chapter 5 and is kept separate from them. Chapter 7 draws conclusions and makes recommendations. Chapter 8 sets out future scope, including the extractive approach that would eliminate the residual position effect by construction.

Chapter 2 Literature Review

2.1 Overlap-Based Automatic Metrics

The earliest widely adopted automatic metrics for generated text, BLEU and ROUGE, measure n-gram overlap against one or more reference texts. BLEU was designed for machine translation and rewards precision of n-grams against a reference; ROUGE was designed for summarisation and emphasises recall. Both became standard because they are cheap, deterministic and correlate acceptably with human judgement on the tasks they were designed for.

They carry an assumption that does not transfer to open-ended question answering: that a good output resembles a specific reference output. For a question admitting many correct phrasings - and most open-ended questions do - overlap with any single reference is a weak proxy for quality. A correct answer phrased differently from the reference scores poorly; an incorrect answer that happens to reuse the reference's vocabulary scores well.

For this project the objection is more fundamental still. There is no reference. The methodology is label-free by construction, so a reference-based metric is not merely imprecise here, it is inapplicable.

2.2 Embedding-Based Metrics

Embedding-based metrics improve on surface overlap by comparing contextual representations rather than tokens. BERTScore computes a similarity between the contextual embeddings of candidate and reference tokens, which tolerates paraphrase: a correct answer using different words is no longer penalised simply for using them.

Sentence-BERT provides the sentence-level embeddings that make such comparisons efficient at scale, by fine-tuning a transformer with a siamese objective so that sentence vectors can be compared directly with cosine similarity rather than requiring a cross-encoder pass per pair.

These remain reference-based, and therefore still require a reference answer this project does not have. Embedding similarity is nevertheless used here, but in two restricted roles that do not require one: as a relevance signal between the question and the answer, and as an agreement signal between the two independently generated candidate answers.

The distinction between "similarity to a reference" and "similarity between two candidates" is not a technicality, and an early version of this project violated it. Section 5.4 documents

what happened when a similarity signal was computed against the combined final answer: because the same model both wrote one candidate and drafted the final answer, the metric measured self-similarity and rewarded it. The signal was removed.

2.3 LLM-as-a-Judge

Using a strong language model to grade the outputs of another has become the dominant evaluation strategy for open-ended generation. Work on MT-Bench and Chatbot Arena established that model judges correlate reasonably well with human preference at the aggregate level, which is what made the approach credible enough to adopt widely.

Subsequent work refined the pattern. G-Eval uses chain-of-thought prompting with a form-filling paradigm to obtain more stable numeric scores, on the observation that asking a model to reason before scoring reduces variance relative to asking for a bare number. Prometheus demonstrated that an open, comparatively small model can be fine-tuned on evaluation rubrics to approach the grading behaviour of a much larger proprietary one, which is directly relevant to any locally hosted deployment: it establishes that a capable judge need not be a frontier model.

This project adopts the structured-rubric element of that line of work. Both evaluators score against an explicit eight-criterion rubric with a justification required per criterion, rather than producing a holistic preference. Structured scoring is what makes two independent evaluators comparable, and therefore what makes their agreement a usable signal.

2.4 Documented Biases of Model-Based Evaluators

The same literature that established LLM-as-a-judge also documents its failure modes, and these are the direct motivation for the bias controls described in Section 3.9 and the measurements in Section 5.8.

- Position bias: evaluators favour candidates presented in a particular slot, independent of content. The effect is reproducible and large enough to reverse a verdict, which is why presentation order must be randomised rather than fixed.
- Verbosity bias: longer answers are preferred, independent of whether the additional length adds information. Any claimed quality effect must therefore be tested against length as a competing explanation.

- Self-enhancement bias: evaluators prefer text generated by themselves or by models of their own family. This is the reason no model in this system both generates and evaluates.

The practical consequence is that an evaluation architecture cannot be assumed to be neutral because it was intended to be. Neutrality is an empirical property and must be measured. Chapter 5 measures it, and finds that this project's own first design was not neutral.

2.5 Multi-Agent Orchestration

Frameworks such as AutoGen popularised the decomposition of a task across several cooperating model-backed agents with distinct roles, communicating through structured messages. The argument is twofold: specialised roles with narrow prompts outperform a single model asked to do everything at once, and disagreement between agents is itself informative.

This project adopts the role decomposition and rejects one common convenience of these frameworks: reusing the same underlying model for several roles. Reuse is attractive operationally - fewer model weights to hold, fewer downloads, less memory pressure - and it is the default in many published examples. But if the generator and the judge are the same model, the evaluation is not independent, and the architecture quietly reintroduces exactly the self-preference bias the literature warns about.

Section 5.8 quantifies what that convenience costs when it is measured rather than assumed away. The cost was 18.1 percentage points of preference for the model's own output, which is not a rounding error.

2.6 Retrieval-Augmented Generation and External Grounding

Retrieval-Augmented Generation reduces hallucination by conditioning generation on retrieved documents rather than relying solely on parametric knowledge. It is relevant to this project in two ways.

First, as an optional input mode: users may upload documents, which are chunked, embedded and retrieved by similarity at query time. Second, and more importantly, as the conceptual basis for the external fact-checking signals. The system queries Wikipedia and Wikidata as two structurally different encyclopedia sources - one prose, one a structured knowledge graph - so that corroboration comes from outside the model family rather than from another model's opinion.

The choice of two structurally different sources is deliberate. Two prose sources would share failure modes; a prose source and a structured source fail differently, so agreement between them is stronger evidence than agreement between two of a kind.

2.7 Threshold Selection and Diagnostic Statistics

Choosing an operating point on a continuous score is a problem with a mature literature in diagnostic testing, and this project borrows from it directly rather than inventing a procedure.

The receiver operating characteristic curve plots sensitivity against one minus specificity across all candidate cutoffs, and the area under it summarises whether the score discriminates at all, independent of any particular cutoff. Youden's J statistic, defined as sensitivity plus specificity minus one, is the standard criterion for selecting a single optimal cutoff from such a curve, corresponding geometrically to the point of maximum vertical distance from the diagonal.

The bootstrap provides a confidence interval for that optimum by resampling the data with replacement and recomputing the optimum in each resample, which answers a question a single point estimate cannot: whether the chosen cutoff is stable or an artefact of this particular sample.

One further element is required here and is not part of the standard recipe. The data in this project are not a simple random sample: they arise from a stratified design in which sub-threshold answers are observed with certainty and above-threshold answers only with a small probability. Inverse-probability-of-sampling weighting, standard in case-control analysis, corrects for this. Section 5.7 explains why omitting it produced a nonsensical result, and Section 5.10 records that the error was caught before it was reported.

2.8 Self-Consistency and Ensemble Approaches

A related line of work improves reliability by sampling a model several times and aggregating, rather than by introducing a second evaluator. Self-consistency decoding samples multiple reasoning paths and takes a majority vote over the final answers, on the observation that correct reasoning converges while errors diverge.

The architecture in this project is an ensemble of a particular kind, and the difference from self-consistency is worth stating because it is easy to conflate the two. Self-consistency samples one model repeatedly, so its diversity is sampling noise around a single model's

beliefs; a systematic error in that model appears in every sample and the majority vote entrenches it. This system samples two different models from different families, so a systematic error in one is not shared by the other, and disagreement between them is informative rather than merely noisy.

That difference is what licenses treating cross-model agreement as a corroboration signal at all. Agreement between two samples of the same model would carry almost no information about correctness. The corpus bears out the diversity: mean pairwise BERTScore between the two candidates is 0.220, reported in Section 5.6, which is low enough to confirm the two generators are genuinely producing different text rather than paraphrasing a shared prior.

2.9 Calibration and Uncertainty

The confidence score produced by this system invites comparison with the calibration literature in machine learning, where a model is said to be calibrated if its stated confidence matches its empirical accuracy - of all predictions made with 70 per cent confidence, 70 per cent should be correct.

That standard cannot be applied here, and the reason is instructive rather than merely inconvenient. Calibration in that sense requires ground-truth correctness to compare the confidence against, which is exactly what this project does not have. The score produced here is therefore not a calibrated probability of correctness, and is deliberately never described as one.

What it is instead is a ranked quality signal with a calibrated decision threshold. The distinction matters for how the output should be read: a score of 0.80 means this answer is better supported than one scoring 0.60, not that it is 80 per cent likely to be true. The threshold is calibrated in the diagnostic-testing sense of Section 2.7 - an operating point chosen to balance sensitivity against specificity on an observable outcome - not in the probabilistic sense.

Establishing genuine calibration would require the human-annotated study proposed in Chapter 8, which is the shortest path from the relative claims this report makes to the absolute ones it declines to make.

2.10 Comparative Summary

Approach	Needs reference?	Needs human labels?	Needs external API?	Used here
BLEU / ROUGE	Yes	No	No	No - no reference exists
BERTScore vs reference	Yes	No	No	No - no reference exists
BERTScore between candidates	No	No	No	Yes - agreement signal
Human preference rating	No	Yes	No	No - no annotation budget
Commercial LLM judge	No	No	Yes	No - offline constraint
Local LLM judge, rubric-based	No	No	No	Yes - Verifier and Judge
External encyclopedia check	No	No	No (public, free)	Yes - Wikipedia, Wikidata

Table 2.1: Evaluation approaches and their applicability to this project

2.11 Identified Gap

The literature establishes that model-based evaluation works at aggregate scale, and that it is biased in known directions. What is comparatively sparse is a complete, reproducible account of building such a system under a hard no-ground-truth, no-external-API constraint, and then turning the measurement apparatus on itself to quantify its own bias with the confounds properly controlled.

Most reported bias studies examine a judge in isolation, on a curated benchmark, with the biases known in advance. This project examines an evaluator embedded in a working pipeline, on a corpus the pipeline generated itself, where the bias was not suspected until it was measured - and where the measurement changed the architecture.

The specific contributions against that gap are listed in Section 1.7 and demonstrated in Chapter 5.

Chapter 3 System Architecture

3.1 Design Principles

Three principles constrain the architecture, and most of the specific decisions in this chapter follow from one of them.

- Separation of concerns between generation and evaluation. No model both produces an answer and judges or merges it. This is the principle that the original design violated at the combiner, and Section 5.8 reports the cost.
- Blindness. Evaluators never learn which system produced which candidate, and the mapping of candidates to prompt positions is randomised per question.
- Plurality of evidence. The quality decision rests on six signals of different kinds - a model's rubric judgement, geometric similarity, cross-model agreement, inter-evaluator agreement, and two external encyclopedia checks - so that no single failing signal can dominate the outcome.

3.2 Pipeline Overview

The pipeline executes six stages per question. Stages 1 and 2 generate two candidate answers independently. Stage 3 applies the Verifier. Stage 4 computes semantic and external signals. Stage 5 applies the Judge. Stage 6 synthesises the final answer from the best attempt. Between stages 5 and 6 the weighted confidence score is computed and the regeneration gate evaluated; if the gate fails, control returns to stages 1 and 2 for a further attempt.

Stages 3 and 4 run concurrently with one another, as do stages 1 and 2, because they have no data dependency. Stage 5 runs strictly after stage 3 despite also being independent of it, for reasons of memory rather than correctness; Section 4.4 explains.

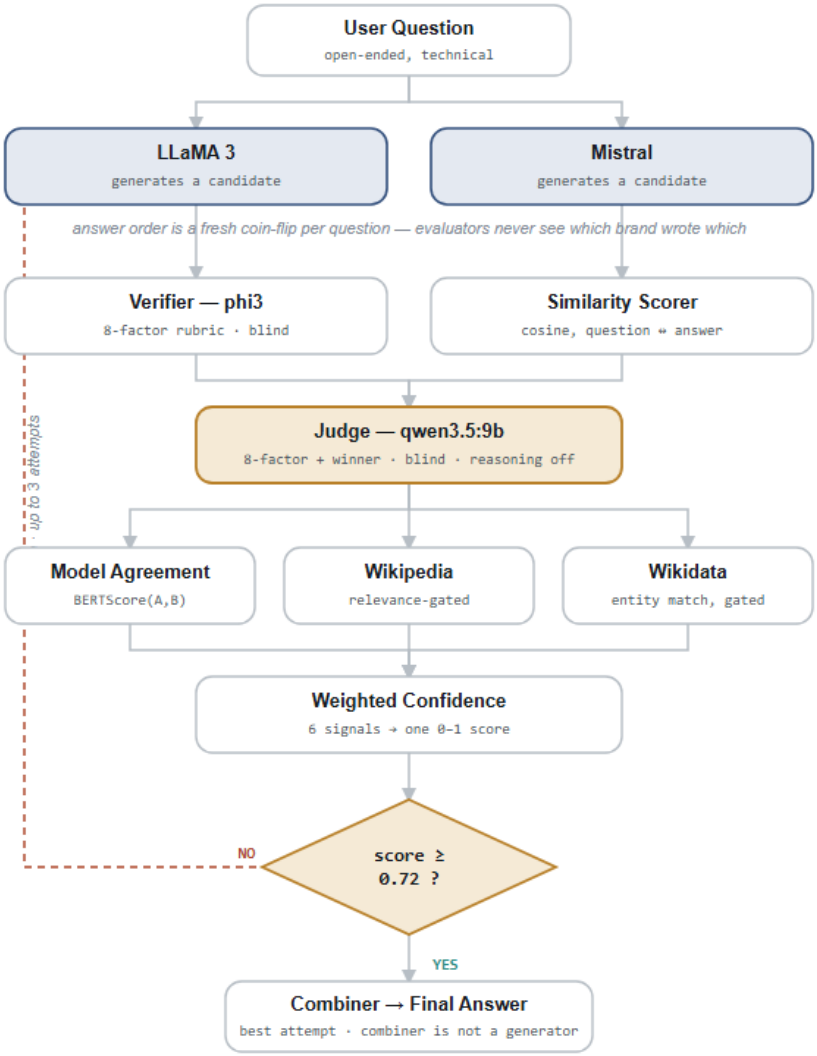


Figure 3.1: Six-stage pipeline architecture, showing the regeneration loop and the signals feeding the confidence score

3.3 Agent Roles and Model Assignment

Each role is served by a deliberately chosen model. The two generators are different model families, so that their errors are less likely to correlate - two models from the same family trained on overlapping data tend to be wrong in the same places, which would make their agreement worthless as a signal. The Verifier and the Judge are each distinct from the generators and from each other. The Combiner is distinct from every generator, for reasons established empirically in Section 5.8 rather than assumed at design time.

Stage	Agent	Model	Temp.	Max tokens	Responsibility
1	Generator A	llama3	0.7	600	Produce a candidate answer
2	Generator B	mistral	0.7	600	Produce an independent candidate

Stage	Agent	Model	Temp.	Max tokens	Responsibility
3	Verifier	phi3:latest	0.1	700	Score both candidates on the rubric
4	Similarity Scorer	all-MiniLM-L6-v2	—	—	Embedding relevance and agreement
5	Judge	qwen3.5:9b	0.1	900	Score both and declare a winner
6	Combiner	gemma2:9b	0.4	1000	Synthesise the final answer

Table 3.1: Agent roles, models and decoding parameters

The temperature assignments follow the role. Generators run at 0.7, because diversity between the two candidates is the point - two identical answers would make the comparison vacuous. Evaluators run at 0.1, because scoring should be as close to deterministic as the runtime allows; an evaluator that returns a different score for the same input on consecutive calls cannot support any claim about the system. The Combiner sits between at 0.4: synthesis needs enough freedom to produce fluent connective prose, but not enough to invent.

The Judge runs with internal reasoning disabled. With reasoning enabled, qwen3.5:9b consumes its entire generation budget on an internal deliberation and returns an empty response on long rubric prompts. The failure is silent in the worst way: the pipeline receives a well-formed but empty reply rather than an error, so the symptom is missing scores rather than a crash. Disabling reasoning resolved it.

3.4 The Eight-Factor Rubric

Both the Verifier and the Judge score each candidate answer against the same eight criteria, each on a scale of zero to ten, for a maximum of eighty points per answer. Using an identical rubric for two independent evaluators is what makes their agreement a meaningful signal: if they were scoring different things, agreement would mean nothing.

Criterion	What it measures	Typical failure it catches
Factual Correctness	Whether the stated facts are accurate	Confident but wrong dates, figures, names
Question Relevance	Whether the answer addresses what was asked	Answering an adjacent, easier question
Context Relevance	Consistency with supplied reference material	Ignoring or contradicting provided documents

Criterion	What it measures	Typical failure it catches
Faithfulness	Whether claims are supportable rather than invented	Plausible but unsupported specifics
Completeness	Whether the answer covers the question adequately	Addressing one half of a two-part question
Hallucination-Free	Absence of fabricated specifics	Invented citations, statistics, quotations
Clarity	Structure and readability	A correct answer the reader cannot follow
Consistency	Absence of internal self-contradiction	Asserting X and later implying not-X

Table 3.2: The eight-factor evaluation rubric

Each criterion requires a one-sentence justification alongside its numeric score. This is not for display: requiring a justification measurably reduces the tendency to assign the same score to every criterion, which is the degenerate behaviour that disqualified one candidate judge model in Section 5.3.

Rubric totals are recomputed from the individual criterion scores rather than taken from the model's own stated total. This is not defensive programming for its own sake. Evaluator models were observed stating totals that did not match the sum of the scores they had just produced, in one case reporting a total of 55 against a stated maximum of 40. Arithmetic is not delegated to the model.

3.5 Semantic and Agreement Signals

Two signals are computed without any language model. Semantic similarity is the cosine similarity between the embedding of the question and the embedding of the answer, using all-MiniLM-L6-v2. It measures relevance - whether the answer is about what was asked - and not correctness; a fluent, on-topic, entirely false answer scores well here, which is precisely why it carries only modest weight.

Model agreement is the pairwise BERTScore between the two candidate answers. Two models prompted independently, from different families, arriving at substantively similar answers is weak corroborating evidence that the answer is right; substantive disagreement is evidence that at least one is wrong. The signal is symmetric and does not reference the final answer, which keeps it free of the self-similarity confound described in Section 2.2.

Verifier-Judge agreement measures the concordance between the two evaluators' independent rubric totals for the winning answer. Two independent evaluators agreeing is a stronger signal than either alone. The Judge does not see the Verifier's scores, which is

what makes the agreement informative rather than circular - an earlier design did leak them, and Section 5.4 records the consequence.

3.6 External Fact-Checking

Two external signals ground the evaluation outside the model family entirely. The Wikipedia check extracts salient entities from the question and answer, retrieves the corresponding articles, and scores the overlap between the answer's claims and the retrieved text. The Wikidata check queries the structured knowledge graph for entity-property assertions and tests whether the answer's specific claims are consistent with them.

Both are relevance-gated. If no sufficiently relevant article or entity is found, the signal returns nothing rather than a low score, and its weight is redistributed across the remaining signals. This matters: a question about an obscure topic should not be penalised for the absence of an encyclopedia entry, which would conflate "unverifiable" with "wrong".

3.7 The Weighted Confidence Score

Six signals are combined into a single value in the range zero to one. Each signal is first mapped onto that range - the Judge's rubric total by division by eighty, the others natively - and then combined as a weighted sum.

Any signal that is unavailable for a given answer is dropped and the remaining weights are renormalised proportionally, so the score remains on the same scale regardless of how many signals contributed. Formally, for the set S of available signals with values v and nominal weights w, the score is the sum over S of v times w, divided by the sum over S of w.

Signal	Weight	Kind of evidence
Judge rubric total	0.40	Structured model judgement, normalised from /80
Semantic similarity	0.15	Geometric relevance, question to answer
Wikipedia check	0.15	External prose corroboration
Model agreement	0.10	Cross-model corroboration (BERTScore A vs B)
Verifier-Judge agreement	0.10	Inter-evaluator concordance
Wikidata check	0.10	External structured corroboration

Table 3.3: Confidence score signal weights

The Judge carries the largest single weight at 0.40 because it is the only signal that assesses correctness directly rather than by proxy. It is deliberately kept below one half, so that the

remaining five signals together can outweigh it: a confidently wrong Judge should be outvoted by the combination of poor relevance, cross-model disagreement and failed external checks.

One omission is deliberate and important. No signal referenced against the final combined answer is included. Such signals were found to be systematically biased toward whichever candidate the Combiner had been built from, because for most of this project the Combiner and one generator were the same model. The bias was removed from the scoring by excluding those metrics; it remained in the shipped answer until Section 5.8 measured it.

3.8 Why Six Signals, and Why These Weights

The weights in Table 3.3 are a reasoned prior rather than a fitted optimum, and the reasoning is given here so that a reader can disagree with it specifically rather than in general.

The design starts from a question about failure rather than about accuracy: what does each signal fail to notice? The Judge assesses correctness directly but is a language model, so it fails where models fail - confident fabrication, plausible-sounding specifics, and its own stylistic preferences. Semantic similarity cannot assess correctness at all, but it never mistakes an off-topic answer for an on-topic one, which the Judge sometimes does when the off-topic answer is well written. Cross-model agreement is blind to shared errors between the two generators, but catches the case where one has gone astray on its own. The external checks cannot assess reasoning, but they are the only signals not produced by a language model at all, so they fail independently of every other signal in the set.

No signal is reliable alone, and the failure modes do not overlap. That is the argument for blending, and Section 5.6 tests it: thirteen of the fifteen signal pairs correlate below 0.13, so the signals really are measuring different things rather than restating one measurement six times.

The Judge carries the largest single weight at 0.40 because it is the only signal that assesses correctness directly. It is deliberately held below one half so that the remaining five together can outvote it: an answer the Judge likes, which is off-topic, which the other generator contradicts, and which fails both encyclopedia checks, should not pass the gate on the Judge's say-so.

The external checks total 0.25 between them. That is high for signals available on fewer than half of attempts, and it is deliberate: when an external source does speak, it is the only

evidence in the set that does not originate from a model, and it should count for something. The renormalisation described in Section 3.7 means their absence costs nothing rather than scoring zero.

The two agreement signals take 0.10 each, the lowest weights, because both are indirect. Agreement is evidence about consistency, not correctness, and two models can agree while both being wrong - a failure mode that is more likely, not less, when they share training data.

A systematic weight sweep against the control-group improvement criterion would establish whether this allocation is near-optimal or merely defensible. That sweep has been only partially run and is listed in Chapter 8. The honest position is that the weights are justified by an argument about independence, corroborated by the correlation structure, and not yet optimised.

3.9 The Regeneration Gate

If the weighted confidence score for an attempt falls below the threshold, the pipeline discards nothing but proceeds to generate again, to a maximum of two further attempts. The highest-scoring attempt is retained even if a later attempt scores lower, so regeneration cannot make the shipped answer worse than it would otherwise have been. This is a deliberate asymmetry: the loop can only improve the outcome or leave it unchanged.

The attempt bound exists because the loop would otherwise be unbounded on questions the model set simply cannot answer well - and such questions exist. Three attempts was chosen as the point beyond which, in practice, further attempts produced variation rather than improvement.

The threshold value itself is the subject of Section 5.7 and is not a design parameter chosen here.

3.10 Bias Controls

Two controls operate on every question.

Brand-blind prompting removes any mention of which system produced a candidate from the evaluator prompts. The candidates are presented only as "Answer 1" and "Answer 2". This is necessary but not sufficient, because a model may still infer provenance from style.

Position randomisation assigns the two candidates to the two prompt slots by a fresh coin flip per question, with the true mapping recorded in separate columns so that per-model

statistics remain recoverable afterwards. Without the recorded mapping the randomisation would destroy the very data needed to analyse identity effects; with it, position and identity can be crossed, which is what makes Section 5.8 possible.

It is worth being precise about what randomisation achieves, because the distinction has practical consequences and was initially stated too loosely in this project's own documentation. Randomisation makes a position effect unbiased with respect to model identity. It does not remove the position effect. An evaluator that favours the first slot still favours the first slot; randomisation only ensures that the favour falls equally on both systems across a large corpus. A system claiming to have addressed position bias should state which of the two it has achieved.

3.11 Retrieval Subsystem

An optional retrieval mode allows users to supply documents. Uploaded files are chunked, embedded with the same encoder used elsewhere in the pipeline, and stored in a local vector table. At query time the question is embedded and the most similar chunks retrieved.

A minimum similarity threshold gates retrieval: if the best matching chunk falls below it, no context is injected and the question is answered from parametric knowledge alone. The purpose is to prevent an unrelated document from contaminating an answer merely because it happens to be the only document available. Retrieval decisions - whether context was injected, how many chunks, and at what similarity - are logged per query, which provides the data to calibrate that threshold by the same control-group method used for the confidence threshold. That calibration has not yet been performed and is listed in Chapter 8.

Chapter 4 Implementation

4.1 Technology Stack

- Model runtime: Ollama, serving all six models locally over HTTP on the loopback interface.
- Orchestration: Python with asyncio; LangChain's Ollama binding wraps each agent behind a uniform interface.
- Web service: FastAPI, streaming pipeline progress to the browser over Server-Sent Events.
- Embeddings: sentence-transformers with all-MiniLM-L6-v2, executed on GPU.
- Persistence: SQLite, one row per attempt, retaining every raw signal rather than only the outcome.
- Frontend: dependency-free HTML, CSS and JavaScript consuming the event stream directly.
- Analysis: NumPy, SciPy and Matplotlib for the statistics and figures in Chapter 5.
- Hardware: a single workstation with an NVIDIA RTX 4090 (24 GB VRAM).

4.2 Agent Abstraction

Every agent is an instance of one small class holding a role name, a model identifier, a decoding temperature, a token budget and a system prompt, exposing a single run method that takes a prompt and returns text. Specialisation lives entirely in configuration and prompt, not in code.

This uniformity is what made the experiments in Chapter 5 cheap to run. Swapping the Combiner from one model to another is a one-line change, and the controlled replay described in Section 5.9 - which re-runs the Combiner alone across hundreds of stored records with every other input frozen - is possible precisely because the agent has no hidden state and no dependency on the rest of the pipeline beyond its prompt.

One agent carries an extra flag. Hybrid-reasoning models such as qwen3.5 support an internal deliberation phase that must be explicitly disabled; the abstraction exposes this as an optional parameter which is simply ignored by models that do not understand it.

4.3 Prompt Design

Generator prompts ask for a thorough, well-structured, factually accurate answer and nothing more. They do not mention the existence of a second generator, an evaluator or a rubric; a generator that knew it was being scored against criteria would optimise for the criteria rather than for the answer.

Evaluator prompts present the question and both candidate answers, labelled only as "Answer 1" and "Answer 2", followed by the eight criteria with their definitions and a required output format: a score out of ten and a one-sentence justification per criterion, per answer. The Judge's prompt additionally asks for an overall winner and a short reasoning paragraph.

The Combiner's prompt supplies the question, both answers, the Verifier's output and the Judge's output, and instructs the model to merge the best elements, remove anything the Verifier flagged, introduce no new information, and produce at most 150 words.

One detail of that prompt turned out to matter a great deal and is easy to get wrong. The Verifier and Judge outputs are free text that refers to the answers by label - the Judge's reasoning paragraph names "Answer 1" explicitly. If the answers are presented to the Combiner in a different order from the one the evaluators saw, those labels point at the wrong text. Section 5.10 records that exactly this defect was present in the live service for the whole of the project.

4.4 Concurrency and Memory Constraints

The six models together substantially exceed the 24 GB of available video memory if resident simultaneously. The pipeline therefore runs with a concurrency of one and relies on the runtime to load and evict model weights as stages execute.

An earlier configuration ran the Verifier and the Judge concurrently, since neither depends on the other's output. This produced memory over-subscription stalls: both models' weights were requested at once, the runtime began evicting and reloading in a loop, and throughput collapsed. The two are now deliberately sequential despite the available parallelism, which costs latency and buys reliability.

Stages 1 and 2 do run concurrently, and stages 3 and 4 likewise, because in each case one of the pair is cheap - the second generator shares a loaded family, and the similarity scorer is a 22-million-parameter encoder rather than a billion-parameter model.

4.5 Streaming Interface

A complete pipeline run takes on the order of one to two minutes, and considerably longer under memory contention. A conventional request-response interface would present the user with an unexplained pause of that length, which is unacceptable for a demonstration and unhelpful for debugging.

The service instead emits a typed event as each stage begins and completes. The frontend renders the pipeline filling in progressively: each generator's answer as it arrives, the Verifier's and Judge's rubric breakdowns, each signal's contribution, the weighted score for the attempt, and - where the gate fails - an explicit banner naming the score and the threshold it missed.

The interface is therefore not only a demonstration surface but the primary debugging tool for the pipeline's behaviour. Several of the anomalies investigated in Chapter 5 were first noticed as something looking wrong on this screen rather than in a log file.

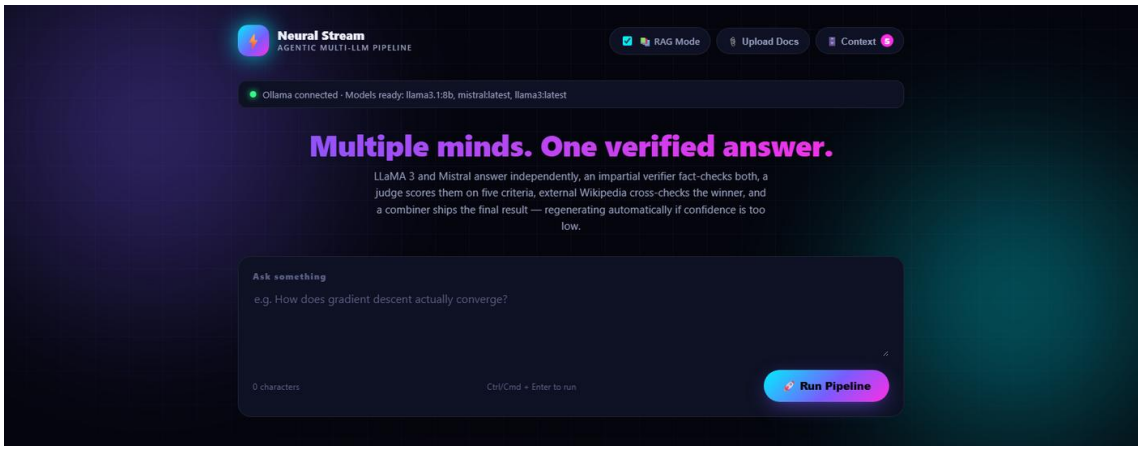


Figure 4.1: Streaming interface: the pipeline renders stage by stage as it executes

4.6 The Event Protocol

The service emits four event types over the Server-Sent Events stream, and the frontend is a state machine over them.

A stage event announces that a numbered stage has begun, carrying the stage index, its display name and a status. A result event carries that stage's output - a generated answer, an evaluator's raw text, a computed signal. A regenerate event fires when the gate rejects an attempt, carrying the score that failed and the threshold it missed, so that the reason is visible rather than inferred from a restarting progress bar. A done event closes the stream with the total elapsed time.

Periodic comment lines are emitted as keep-alives. Without them an intermediary can close a connection that has been quiet for the length of a model call, which on a contended host can exceed a minute.

Streaming the reason for a regeneration rather than only the fact of one turned out to matter for the project rather than only for the user. Several anomalies investigated in Chapter 5 were first noticed as a score that looked wrong next to the threshold it was being compared against.

4.7 Frontend

The frontend is dependency-free HTML, CSS and JavaScript - no framework, no build step. The choice was practical: the page is served from the same process as the API, it has one job, and a build step would have added an obligation without adding a capability.

It renders a stage rail that fills as events arrive, per-answer cards for the two candidates, a breakdown of each evaluator's rubric scores, a chip per confidence signal showing its value and contribution, and a gauge for the weighted score. Each attempt's score is appended persistently rather than replaced, so a question that regenerated twice shows all three scores in sequence and the reader can see whether the loop was helping.

A rolling context drawer exposes the last few queries and their answers, which are injected into subsequent prompts to provide continuity across questions.

4.8 Retrieval Store

Uploaded documents are chunked, embedded with the same encoder used for the similarity signal, and stored in a local table alongside their source filename and offset. Retrieval embeds the question and ranks chunks by cosine similarity.

Only chunks above the minimum similarity threshold are injected, and the decision is logged per query: whether context was used, how many chunks, and the similarity of the best match. The logging exists specifically so that the threshold can later be calibrated by the control-group method rather than left at its inspected value, as described in Chapter 8.

When context is injected it is prepended to both generator prompts and to the Verifier prompt, and the Context Relevance criterion is scored against it. When no context clears the threshold, that criterion falls back to assessing whether the answer stays on topic - which is why the rubric defines it in two parts.

4.9 Data Model

Each attempt is stored as one row. The row carries the question and its index, the attempt number, both candidate answers, the true model-to-position mapping, every per-criterion score from both evaluators for both answers, the declared winner, all six confidence signals, the weighted score, the reason the attempt was regenerated if it was, and the elapsed time. The final attempt for a question additionally carries the combined answer and is flagged as final.

Storing every raw signal for every attempt rather than only the outcome is what makes the analyses in Chapter 5 possible. The combiner bias investigation needed to reconstruct exactly what each evaluator saw and in what order, months after the run completed; a schema that had retained only final answers and final scores would have made that investigation impossible without regenerating the entire corpus.

A subtle defect in how rows were marked final is documented in Section 5.10, because it illustrates a class of bug that silently corrupts aggregate statistics without producing any error.

4.10 Dataset Generation Harness

A batch harness runs the full pipeline over a question corpus and writes one row per attempt. It is restartable: it records which question indices are complete and resumes from the first incomplete one, because a run spanning several days is interrupted in practice by reboots, driver updates and power events.

The harness also implements the control group described in Section 5.7. With a configured probability, an attempt that has already passed the confidence gate is regenerated anyway, purely to produce an observation that the gate would otherwise censor. These rows are marked with a distinct regeneration reason so that the analysis can weight them correctly.

That probability was changed mid-run, from 0.08 to 0.60, to accumulate control observations faster once it became clear how few the lower rate was yielding. The change is innocuous operationally and a trap analytically; Section 5.10 records the error it caused and the correction.

4.11 Operational Incidents

Three classes of failure dominated the run, and all three are specific to orchestrating several large models on one machine rather than to anything about language models. They are

recorded because each cost real time and none is documented in the frameworks' own guidance.

Orphaned inference subprocesses. The model runtime spawns a child inference process per loaded model. On Windows these children survive a normal restart of their parent, because there is no cascade kill, so every manual restart silently left one behind holding video memory. They accumulated across restarts until the next evaluator call wedged for want of VRAM. The symptom - an intermittent hang that got more frequent the longer the day went on - pointed at the models rather than at process hygiene, which is why it took a while to find. The watchdog now enumerates and kills the inference children directly rather than only their parent.

Racing watchdogs. Stopping the monitoring harness did not reliably terminate the underlying shell, so multiple watchdog instances accumulated and competed, each launching its own generator. The result was duplicate work and a corrupted sense of progress. Watchdogs are now killed by process id rather than by asking the harness to stop them.

Silent total stoppage. Three separate gaps occurred - 17 hours, 40 minutes, and four full days - in which every pipeline process had died and only the model runtime remained. The cause was that all of them, including the watchdog, were launched from within an interactive session's process tree, which does not survive terminal closure, session teardown, or the machine sleeping. A keep-awake routine blocked automatic idle sleep but not manual sleep or session loss. Each gap was discovered only when somebody next looked.

The common thread is that a watchdog inside the thing it is watching is not a watchdog. The fix was to move supervision entirely outside the session, into a scheduled task that the operating system runs whether or not anyone is logged into a terminal.

4.12 Resilience

Three separate multi-hour gaps in generation occurred before resilience was addressed, caused by terminal closure, session teardown and the machine sleeping. In each case the harness had stopped and nothing restarted it, and the loss was discovered only when progress was next inspected.

A scheduled task external to any interactive session now checks every five minutes that the required processes are alive and restarts any that are not. The check matches on process

command line rather than on a stored process identifier, so it survives the restarts it performs.

This is unglamorous engineering, and it is reported here because a multi-day unattended experimental run is not achievable without it. The 5,000-question corpus that Chapter 5 rests on would not exist otherwise.

One lesson from operating it is worth recording. When the generator finished, the watchdog continued restarting it every five minutes; each restart re-scanned 5,000 completed questions, found nothing to do, and exited. The behaviour was harmless but wasteful, and it was removed from the watch list once generation completed. A watchdog needs a notion of "done", not only a notion of "running".

Chapter 5 Experiments and Results

5.1 Experimental Setup

A corpus of 5,000 questions spanning ten knowledge domains was generated and processed through the complete pipeline, producing 8,432 attempt records of which 5,000 are final answers. All statistics in this chapter are computed over this corpus unless stated otherwise, and all figures are generated directly from it.

Property	Value
Questions	5,000
Knowledge domains	10
Total attempt records	8,432
Final answers	5,000
First attempts	5,005
Second attempts	2,001
Third attempts	1,426
Mean attempts per question	1.69
Maximum attempts permitted	3
Control-group regenerations	165

Table 5.1: Corpus composition

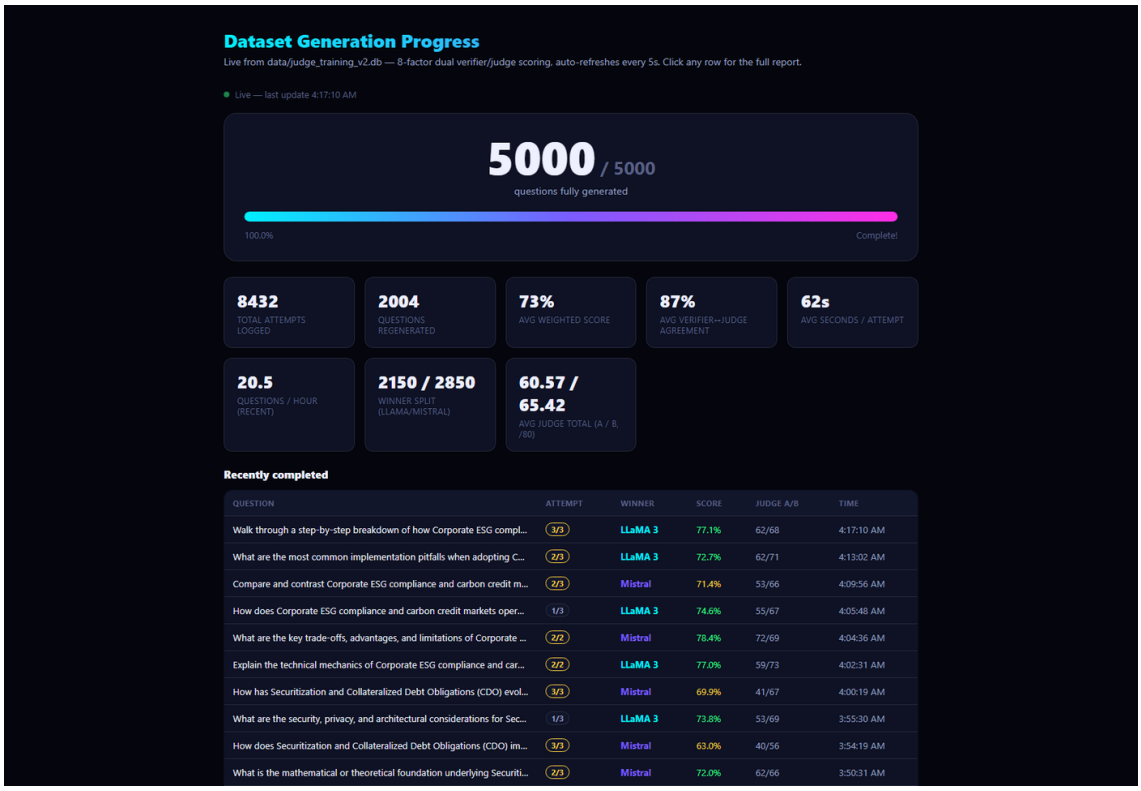


Figure 5.1: Dataset generation monitor at completion of the 5,000-question corpus

5.2 Corpus Characteristics

The distribution of the weighted confidence score is unimodal and moderately tight, with a mean of 0.702 across all attempts and a range from 0.307 to 0.856. The final answers sit visibly to the right of the full attempt distribution, which is the regeneration loop working as intended: weaker attempts are replaced, so the surviving population is better than the population that was generated.

Against the adopted threshold of 0.72, 1,849 of the 5,000 final answers - 37.0 per cent - still fall below it. This is not a failure of the gate. It is the attempt bound being reached: those questions were regenerated up to the permitted maximum and did not clear the threshold, so the best available attempt was shipped. The figure is a useful honesty check on the system, since it quantifies how often the pipeline ships an answer it is not confident in.

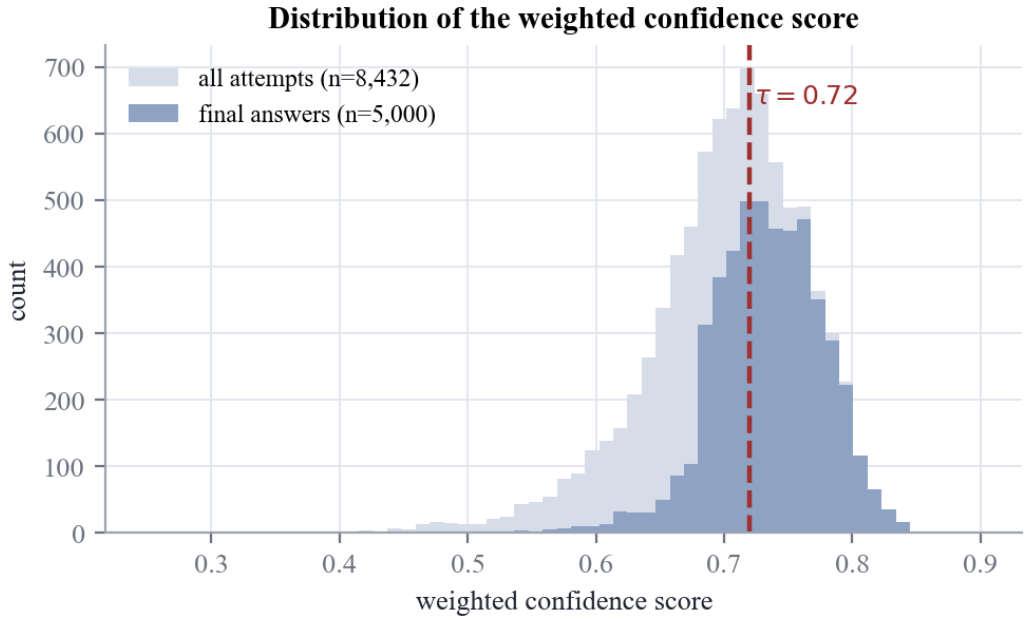


Figure 5.2: Distribution of the weighted confidence score, all attempts against final answers, with the adopted threshold marked

5.3 Regeneration Behaviour

Of the 8,432 attempt records, 4,163 were regenerated because they fell below the confidence gate, 165 were regenerated as control samples despite passing it, and 4,104 were not regenerated at all. The mean number of attempts per question is 1.69, so the regeneration loop roughly doubles the computational cost of the pipeline relative to a single-pass system.

That cost is the price of the quality mechanism, and it is worth stating plainly rather than burying: this system is slower than a single model answering once, by roughly the factor

its attempt distribution implies, plus the cost of four evaluation stages. It buys a calibrated confidence score and an automatic second chance.

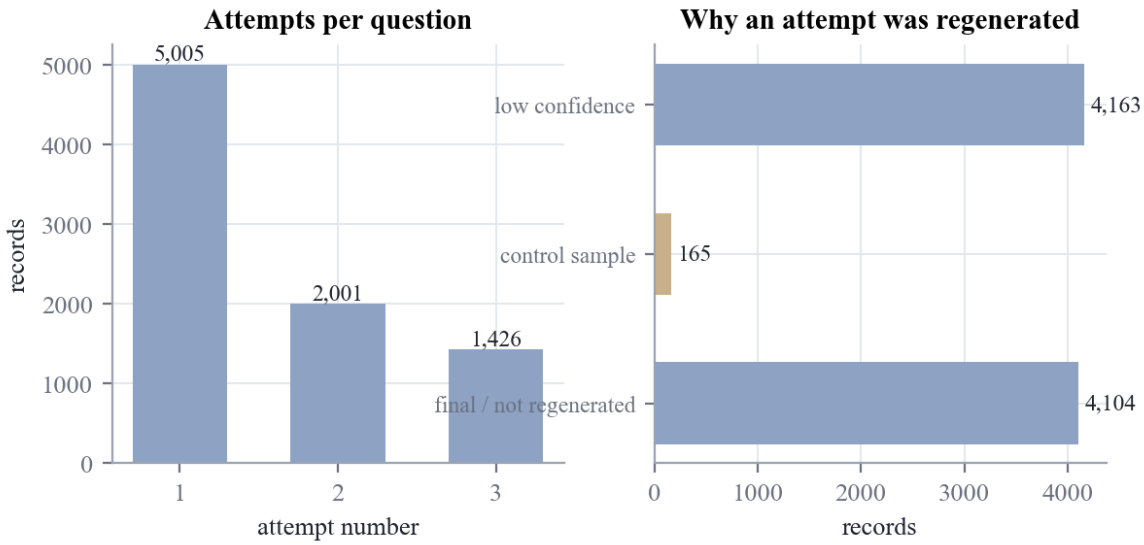


Figure 5.3: Attempts per question, and the reason each regeneration was triggered

5.4 Does Regeneration Improve the Answer?

The regeneration loop is the system's only corrective mechanism, and the report has so far described how often it fires without establishing that it works. Of the 5,000 questions, 2,004 produced two or more attempts, which permits a direct before-and-after comparison on the same question.

Transition	n	Mean change	Median change	Improved
Attempt 1 → attempt 2	2,004	+0.0130	+0.0136	59.3%
Attempt 2 → attempt 3	1,427	+0.0132	+0.0117	—

Table 5.9: Score change across regeneration attempts

Regeneration helps on average, and the effect is consistent across both transitions: roughly +0.013 on a zero-to-one scale each time. In the context of a threshold at 0.72 and a corpus standard deviation of 0.044, that is a modest but real shift - about three tenths of a standard deviation over two retries.

The more important number is the one that is not an average. A second attempt improved on the first in only 59.3 per cent of cases, which means that in 40.7 per cent the retry produced a worse answer than the one it was meant to replace. Regeneration is not a reliable repair; it is a resample from a distribution whose mean is slightly higher.

That finding is what justifies a design decision made earlier on intuition. The pipeline retains the highest-scoring attempt rather than the most recent one, and the corpus shows how much work that rule does: the best attempt was the last attempt in only 56.8 per cent of multi-attempt questions. In the remaining 43.2 per cent, a system that simply shipped its latest retry would have shipped a worse answer than one it had already produced and discarded.

Stated plainly, nearly half the value of the regeneration loop comes not from regenerating but from remembering. This is worth emphasising because the obvious implementation - overwrite and retry until the gate passes or the budget runs out - would have discarded that value silently, and no aggregate statistic would have revealed the loss.

5.5 Judge Selection: Four Models and a Methodology Error

Selecting the Judge took four candidate models and two false starts, and the sequence is reported in full because the criterion that eventually worked is not the one that would be chosen by default.

The first question was whether the ceiling-clustering of Section 5.6 was specific to the incumbent model. Four further format-compatible models were run on an identical prompt and the same real question-answer pair.

Model	Answer A	Answer B
qwen2.5 (incumbent)	~76/80	~76/80
gemma2:9b	72/80	75/80
llama3.1:8b	72/80	73/80
mixtral	72/80	76/80
CompassJudger-1-7B	72/80	75/80

Table 5.12: Five candidate judges on one real question-answer pair

All five models, across four architectures, converged on 72-76 out of 80. That looked like a robust consensus, and treating it as one would have been a methodology error: testing a single strong answer repeatedly only establishes that models agree a good answer is good. It says nothing about whether they can tell a bad answer from a good one, which is the only property that matters for a gate.

The corrected test constructed a deliberately broken answer - factually wrong, internally contradictory, containing irrelevant claims - and presented it alongside a solid one for the

same question. This is the discrimination test that Section 5.7 later applied to the Verifier, and it is the criterion on which the Judge was ultimately chosen.

A head-to-head on twenty real question-answer triples, forty scores per model, then compared the incumbent against the most promising alternative.

Metric	qwen2.5	CompassJudger-1-7B
Mean	71.9/80 (89.9%)	70.0/80 (87.5%)
Standard deviation	3.7	3.8
Min / Max	64 / 80	60 / 73
Scored ≥75	20.0%	0.0%

Table 5.13: Head-to-head on 20 real triples (N=40 scores per model)

CompassJudger-1 never once scored 75 or above across forty real evaluations, where the incumbent reached the maximum of 80 one time in five. Combined with evidence that it still penalised genuinely wrong content, this read as a real calibration improvement, and the Judge was switched.

That decision was wrong, and the reason it was wrong is the most useful finding in this section. Mean and spread are aggregate statistics, and they concealed the failure that mattered. Inspecting per-criterion scores rather than totals showed that CompassJudger-1 was giving all eight criteria the identical value on most answers - 9/9/9/9/9/9/9/9 - with boilerplate justifications that were sometimes word-for-word identical between two different answers.

Condition	n	All-8-identical
Anchored on the Verifier's scores (pre-fix)	40	10%
Blind, rubric v3	5	80%
Blind, rubric v4	4	75%
Blind, rubric v4, generation continued	43	~74%

Table 5.14: CompassJudger-1 flat-scoring rate (all eight criteria identical)

The rate got worse after the independence bug was fixed, not better. While anchored on the Verifier's output the model had been inadvertently inheriting the Verifier's genuine variation; scoring independently exposed that its own default behaviour was to write plausible prose around a single flat number. A bug had been masking a worse problem, and fixing the bug is what revealed it.

The final comparison, at full scale under the production rubric, settled the choice.

Metric	qwen3.5:9b	CompassJudger-1-7B
Mean	65.5/80 (81.9%)	69.3/80 (86.7%)
Standard deviation	9.5	9.3
Min / Max	19 / 78	16 / 73
Scored ≥75	10.0%	0.0%
Flat (all 8 criteria identical)	0.0%	57.5%

Table 5.15: Final judge comparison (N=40 blind evaluations each)

The two standard deviations are nearly identical, and on that statistic alone the models look equivalent. They are not. CompassJudger-1's spread comes from which single flat value it happens to select per question, not from per-criterion analysis; qwen3.5:9b never produced a flat score across forty evaluations. The selection criterion that distinguishes them - per-criterion variance - is not one that appears in any standard evaluation summary, and would have been missed by comparing means.

qwen3.5:9b was adopted, with internal reasoning disabled for the reason given in Section 3.3.

5.6 Calibrating the Rubric: Four Revisions

The eight-factor rubric presented in Section 3.4 is the fourth version. The three it replaced each failed in an instructive way, and the sequence is reported here because the failures were not obvious in advance and the fix that seemed most natural did not work.

The first version gave each criterion a single generic descriptor - "claims are accurate", "well-structured and easy to read" - and relied on the model to map quality onto a zero-to-ten scale. Across the first ~870 records, Judge totals clustered hard against the ceiling: mean 76.3 out of 80, standard deviation 3.4, with 81 per cent of all scores falling in the 75-80 band. The rubric was not discriminating between answers of different quality; it was agreeing that everything was good.

The obvious remedy was tried first and is the most useful result in this section because it failed. The band descriptions were rewritten to be explicitly stricter - "competent/adequate" recalibrated from 7-9 down to 5-6, and an explicit requirement to justify any score of 9 or 10 before awarding it. Eight fresh completions under the new prompt averaged 76.4 out of 80 with the same tight spread. The change had no measurable effect at all.

The conclusion drawn was that abstract calibration language does not override an evaluator model's systematic positivity, even when the instruction to be harsh is explicit. Telling a

model to be stricter is not the same as giving it a basis on which to be stricter. Everything that worked subsequently followed from that distinction.

Version	Change	Outcome
v1 / v2	Generic one-line descriptor per criterion	Mean 76.3/80, SD 3.4, 81% of scores in 75-80 — not discriminating
(interim)	Stricter band language; justification required for 9-10	Mean 76.4/80, same spread — no measurable effect
v3	Narrower bands; must find a specific flaw before awarding ≥7, else cap at 6	Forces an evidence step rather than a judgement
v4	Concrete per-criterion anchors + explicit anti-default rule	Adopted; differentiated per-criterion scores (Table 5.5)

Table 5.11: Rubric revisions and their effect on Judge behaviour

Version 3 changed the shape of the task rather than its wording. Instead of asking for a calibrated number, it required the model to actively search for a specific flaw before awarding any score of seven or above, and to cap the criterion at six if it found one. This converts scoring from an impression into a search with an observable outcome, which a language model is considerably better at.

Version 4 addressed a different failure that surfaced immediately after the independence fix of Section 5.5. The Judge's total came out as exactly 72 on three unrelated questions in the first five records - CRISPR, quantum entanglement, and a machine-learning comparison. A value recurring across unrelated content is the signature of a comfortable default rather than content-driven scoring.

The diagnosis was that all eight criteria shared the same vague standard, which gives a model little basis to distinguish a four from a six on any one of them. Version 4 rewrote every criterion with a concrete, criterion-specific differentiator - Completeness's mid-band became "covers the obvious parts but skips a sub-part, an edge case, or a caveat a subject-matter expert would expect" rather than a generic quality statement - and added an explicit anti-default rule. The per-criterion variation reported in Table 5.5 is the result.

One procedural consequence is worth recording. Because the scoring regime changed, roughly 435 records generated under the lenient rubric were archived rather than merged, and generation restarted from scratch. Mixing two scoring standards in one corpus would have made every aggregate statistic in this chapter uninterpretable, and the cost of regenerating was far lower than the cost of a dataset nobody could reason about.

5.7 An Evaluator Coupling Error

A more instructive failure concerned the evaluators' independence itself. Checking whether the Verifier and the Judge were really scoring separately, a query over the live database returned something worse than mere similarity: 32 of 37 records - 86 per cent - had Judge totals matching Verifier totals exactly, digit for digit, across all eight criteria for both answers. Two genuinely independent evaluations do not land on the same sixteen numbers that often.

The cause was in the Judge's prompt builder, which received the Verifier's full raw output - including its per-criterion numbers - and inserted it under the heading "Verifier's independent scores (a third-party opinion, not ground truth)". The system prompt instructed the Judge to form its own judgement, but at temperature 0.1 with identical rubric anchoring, an instruction not to be influenced by a number placed directly in front of the model is not an effective control. The Judge was transcribing rather than judging.

The downstream consequence was that the Verifier-Judge agreement signal agreed with itself by construction, inflating the confidence score wherever the Judge was confident - including wherever the Judge was confidently wrong.

The clue was not a crash or an anomaly in the data. The clue was that the signal performed suspiciously well: it correlated with the Judge far more tightly than an independent second opinion plausibly could. Tracing where its inputs came from showed the coupling. It was removed from the score entirely.

This is the first of four measurement errors catalogued in Section 5.10, and it established the habit that found the other three: a result that looks too clean is a reason to check the derivation, not a reason to celebrate.

5.8 Verifier Discrimination

For the Verifier's scores to be meaningful, the Verifier must be capable of distinguishing good answers from bad ones. An evaluator that assigns high scores indiscriminately would produce a confidence signal that looks healthy and means nothing.

This was tested directly by presenting deliberately corrupted answers alongside intact ones. The Verifier scored intact answers in the range 77 to 80 out of 80, and corrupted answers between 0 and 8 - a separation of roughly 74.5 points on an 80-point scale.

The result matters for interpreting the bias findings that follow. It establishes that the evaluators discriminate strongly when there is something to discriminate, so a subsequent finding that they nevertheless exhibit systematic preference cannot be dismissed as the behaviour of an instrument that simply does not work.

5.9 Per-Criterion Score Behaviour

Because both evaluators score the same eight criteria, the corpus permits a direct comparison of how each uses the rubric. The table reports the mean and standard deviation of each criterion for the winning answer across all 5,000 final records.

Criterion	Judge	Verifier	Gap
Factual Correctness	8.02 ± 1.31	8.58 ± 0.69	+0.56
Question Relevance	9.05 ± 0.73	9.12 ± 0.45	+0.07
Context Relevance	8.38 ± 0.67	9.25 ± 0.64	+0.87
Faithfulness	8.50 ± 1.04	9.21 ± 0.53	+0.71
Completeness	7.87 ± 0.93	8.74 ± 0.58	+0.87
Hallucination-Free	8.97 ± 1.14	9.93 ± 0.35	+0.96
Clarity	8.74 ± 0.56	8.94 ± 0.52	+0.20
Consistency	9.21 ± 0.61	9.18 ± 0.90	−0.03

Table 5.5: Per-criterion scores for the winning answer (mean ± SD, out of 10)

Three patterns are visible and none was designed in.

First, the Verifier is systematically more lenient than the Judge. It scores higher on seven of eight criteria, by an average of just over half a point. The two evaluators therefore agree substantially on the ranking of answers while disagreeing on their absolute level - which is the right way round for the architecture, since the Judge's total feeds the confidence score and the Verifier's contributes only through agreement.

Second, the Judge discriminates more. Its standard deviations are larger on six of eight criteria, most markedly on Factual Correctness (1.31 against 0.69) and Hallucination-Free (1.14 against 0.35). A scoring criterion is only useful to the extent that it varies, and the Judge's scores vary more.

Third, and most usefully, the Verifier's Hallucination-Free criterion is close to degenerate: a mean of 9.93 with a standard deviation of 0.35 means it awards full marks almost always. That is the same degenerate behaviour that disqualified a candidate judge model in Section 5.3, appearing here on a single criterion of an otherwise serviceable evaluator. The practical consequence is that the Verifier contributes essentially no information about hallucination,

and the system's hallucination signal rests on the Judge alone. This was not known before the corpus was analysed, and it is a weakness rather than a feature.

The hardest criterion for the Judge is Completeness, at 7.87 - the lowest mean of the eight. Where this system's answers fall short, they fall short most often by being incomplete rather than by being wrong.

5.10 Inter-Evaluator and Cross-Model Agreement

Verifier-Judge agreement averages 0.837 with a median of 0.850 across the corpus. Two independently prompted models, from different families, scoring the same answers against the same rubric without sight of each other's scores, agree to that degree. This is the single strongest piece of evidence that the rubric is being applied consistently rather than idiosyncratically.

Cross-model agreement between the two candidate answers, measured as pairwise BERTScore, averages only 0.220. The two generators produce substantively different text on most questions. For the diversity argument of Section 3.3 this is good news: two models that said the same thing would make the comparison vacuous. For the signal itself it is a limitation, since a signal clustered at the low end of its range has little dynamic range to contribute, and its 0.10 weight therefore does less work than the nominal allocation suggests.

5.11 Are the Six Signals Independent?

The confidence score combines six signals on the premise that they carry different evidence. If they were strongly correlated the blend would be theatre - six numbers measuring one thing, with the weights creating an illusion of breadth. The corpus allows the premise to be tested directly.

	Judge	**Simil.**	**Agree.**	**Ver-Jdg**	**Wiki**	**Wikidata**
Judge rubric	1.00	−0.07	0.12	0.61	0.05	−0.02
Similarity	−0.07	1.00	−0.00	−0.01	−0.08	−0.08
Model agreement	0.12	−0.00	1.00	0.25	0.01	−0.04
Verifier–Judge	0.61	−0.01	0.25	1.00	0.04	−0.05
Wikipedia	0.05	−0.08	0.01	0.04	1.00	0.11
Wikidata	−0.02	−0.08	−0.04	−0.05	0.11	1.00

Table 5.10: Pearson correlation between confidence signals (5,000 final answers)

The premise holds. Of the fifteen distinct signal pairs, thirteen have an absolute correlation below 0.13, and most sit within a few hundredths of zero. The signals are close to mutually orthogonal, which is the strongest possible justification for blending them: each contributes evidence the others do not.

Three entries deserve individual comment.

The Judge and the Verifier-Judge agreement signal correlate at 0.61, by some distance the largest value in the table. This is partly by construction - the agreement measure is computed against the Judge's own total, so the two cannot be independent - and it means the confidence score carries roughly 0.50 of effective weight on the Judge's opinion rather than the nominal 0.40. That is a design observation rather than an error, but it is not what the weight table implies, and it is recorded here rather than left for a reader to derive.

Wikipedia and Wikidata correlate at only 0.11 despite both being external encyclopedia checks on the same answer. Two sources of the same kind would be expected to agree far more strongly. The low value vindicates the choice of structurally different sources - one prose, one a knowledge graph - because they evidently fail on different questions, which is precisely what makes their combination informative.

Semantic similarity is essentially uncorrelated with everything, including the Judge at −0.07. It measures whether the answer is about the question, which no other signal measures, and it is the clearest case in the table of a signal earning its weight by being different rather than by being strong.

Against the composite score, the signals rank Judge 0.73, Wikipedia 0.61, Verifier-Judge 0.57, Wikidata 0.29, model agreement 0.27 and similarity 0.25. Wikipedia's influence is notably higher than its 0.15 weight suggests, because on the 44 per cent of attempts where it is available it varies widely, whereas several other signals cluster.

5.12 Availability of the External Signals

The two external encyclopedia checks carry a combined nominal weight of 0.25, a quarter of the confidence score. How often they actually contribute is a separate question, and the answer is a genuine limitation of the system as built.

Signal	Available	Unavailable	Availability	Mean when present
Wikipedia	3,745	4,687	44.4%	0.606
Wikidata	3,830	4,602	45.4%	0.472

Table 5.6: External signal availability across 8,432 attempts

Both external checks are absent more often than they are present. When no sufficiently relevant article or entity is found the signal returns nothing and its weight is redistributed, which is the correct behaviour - scoring an unverifiable claim as wrong would be worse - but it means that for the majority of answers the confidence score is carried by the Judge, semantic similarity and the two agreement signals alone.

This qualifies the claim that the architecture grounds its judgements outside the model family. It does so for roughly 45 per cent of attempts. For the remainder, every contributing signal originates from a language model or an embedding of one, and the external corroboration that distinguishes this design is simply not available. Improving retrieval coverage is therefore a higher-value direction than re-weighting the signals that already exist.

5.13 Position Skew in the Judge

The Judge declared the answer in the second prompt slot the winner in 3,345 of 5,000 final records, against 1,655 for the first slot - 66.9 per cent against 33.1 per cent.

Because the assignment of models to slots is randomised per question, this cannot be a quality difference between the two generators. It is a position effect in the Judge, and a large one. Decomposed by true model identity rather than slot, the same records give a much smaller preference of roughly 57 to 43 in Mistral's favour, which is the genuine model difference such as it is.

The practical significance is that the raw winner column is close to useless as a statement about which model is better, and would be badly misleading if reported as one. It is retained because the identity mapping is stored alongside it, which allows the position component to be removed after the fact. A system that had not recorded that mapping would have 5,000 records of an uninterpretable verdict.

This finding is also what motivated the bias investigation of Section 5.8. An evaluator with a documented 34-point position skew makes it implausible that a downstream consumer of its verdicts is unbiased, and that suspicion proved correct.

5.14 Comparing the Two Generators

With the position mapping recorded, the corpus supports a comparison of the two generators by true model identity rather than by prompt slot. The result is more interesting

than a simple ranking, because the two evaluators do not agree about whether a difference exists at all.

Measure	LLaMA 3	Mistral	Difference
Judge rubric total (/80)	61.66	64.34	+2.68 to Mistral
Verifier rubric total (/80)	72.69	72.91	+0.22 to Mistral
Mean answer length (words)	366	317	+49 to LLaMA 3
Judge declared winner	2,150 (43.0%)	2,850 (57.0%)	z = −9.90

Table 5.7: The two generators compared by true identity (5,000 final records)

The Judge sees a clear and statistically unambiguous difference. It scores Mistral 2.68 rubric points higher on average and declares it the winner in 57.0 per cent of questions against 43.0 per cent - a deviation from parity with a z-statistic of −9.90, which on 5,000 records is not attributable to chance by any margin.

The Verifier, scoring the same 5,000 pairs of answers against the same eight criteria, sees a difference of 0.22 points. That is a twelfth of the gap the Judge reports, and on the scale of a rubric scored out of eighty it is indistinguishable from no difference at all.

Two independent evaluators disagreeing about whether a difference exists is a more informative result than either evaluator's verdict taken alone. There are two readings and the data here do not separate them. The difference may be real but subtle, with the Judge more sensitive to it than the Verifier - consistent with the Judge's generally larger score variance reported in Section 5.5. Or the Judge's preference may be a model-specific artefact that the Verifier simply does not share, in which case the 57 to 43 win rate measures a property of qwen3.5:9b rather than a property of Mistral.

This report does not claim to resolve it, and the ambiguity is reported rather than collapsed into whichever reading is more convenient. Resolving it would require a third independent evaluator, or the human study proposed in Chapter 8. What can be said is that any statement of the form "Mistral outperforms LLaMA 3 on this corpus" rests entirely on one evaluator's judgement and is not corroborated by the other.

The length figures bear on the earlier bias analysis. LLaMA 3 writes 15 per cent longer answers, which is exactly the kind of difference that could drive a verbosity effect - and it is the competing explanation tested and excluded in Section 5.9.

5.15 Computational Cost

A single attempt - two generations, two evaluations, the embedding work and the external lookups - takes 62.3 seconds on average, with a median of 58.5 seconds. The closeness of mean and median indicates a reasonably symmetric distribution without a heavy tail of pathological cases.

Across 8,432 attempts the corpus represents approximately 145.8 GPU-hours of computation on a single RTX 4090. At a mean of 1.69 attempts per question, a question costs roughly 105 seconds end to end.

Two observations follow. First, the regeneration loop is the dominant cost multiplier: eliminating it would reduce total compute by about 40 per cent, which is the price paid for the quality mechanism. Second, the Judge is the dominant per-attempt cost, being the largest model invoked and invoked on every attempt - which is what makes the distillation direction in Chapter 8 attractive, since replacing it with a small regressor would cut evaluation to milliseconds.

These figures also explain why the corpus took several days of wall-clock time and why the resilience work described in Section 4.7 was not optional.

5.16 Threshold Calibration: the Censoring Problem

The regeneration threshold cannot be calibrated naively, and the reason is structural rather than statistical fussiness.

The quantity that ought to determine the threshold is: for an answer scoring x, would regenerating it produce a better answer? But answers above the operating threshold are never regenerated, so for the entire region above the current threshold, that quantity has never been observed. The data are censored precisely where the decision is most delicate. Fitting a threshold to the visible data alone estimates the wrong quantity, and does so in a direction that tends to confirm whatever threshold is already in force.

The remedy adopted here is a control group. A random fraction of answers that have already passed the gate are regenerated anyway, purely for measurement. This yields unbiased observations above the threshold at a modest cost in computation - 165 such regenerations over the corpus.

The control group introduces a complication of its own. Sub-threshold answers are regenerated with certainty, while above-threshold answers are regenerated only with the

control probability. The resulting sample is stratified, not random, and treating a control-sampled row as equally representative as a census row undercounts the above-threshold population by the reciprocal of the control probability. Inverse-probability-of-sampling weighting corrects for this, with each control row carrying weight equal to one over the probability in force when it was generated.

5.17 Threshold Calibration: Three Independent Routes

Three analyses were applied to the corrected data. They are independent in the sense that they rest on different assumptions and could in principle disagree; that they do not is the substance of the result.

First, a two-proportion significance test compared the improvement rate between confidence bands. The rate differs across bands beyond what sampling noise explains, at p = 0.011. This establishes that the confidence score carries information about whether regeneration will help - a precondition for any threshold being meaningful at all.

Second, a receiver-operating-characteristic analysis treated the attempt's confidence score as a predictor of whether a regeneration would improve the answer. The area under the curve is 0.629. This is modest - a perfect predictor scores 1.0 and a coin flip 0.5 - and it is reported as it is rather than framed favourably. The score discriminates, but weakly. Maximising Youden's J places the point optimum at t = 0.70, with sensitivity 54.6 per cent and specificity 64.4 per cent.

Third, a bootstrap over 2,000 resamples gave a median optimal cutoff of 0.72, a modal value of 0.70, and a 95 per cent confidence interval of [0.69, 0.75].

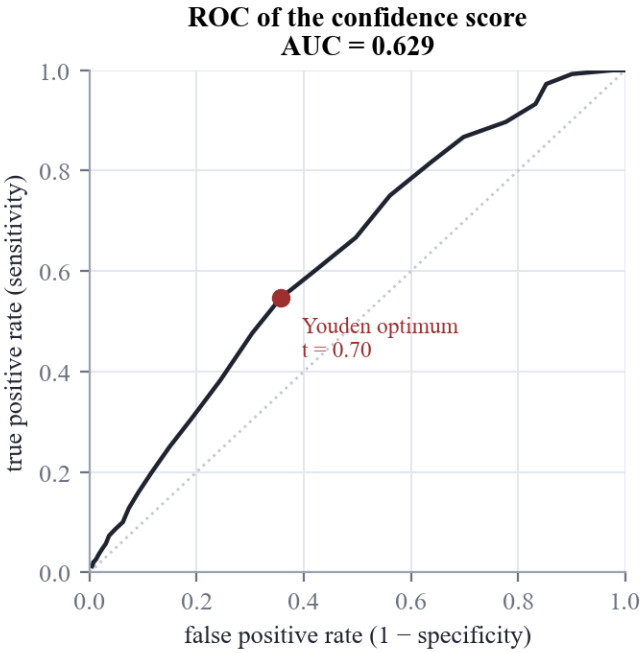


Figure 5.4: Receiver operating characteristic of the confidence score against observed improvement, sampling-corrected

5.18 Threshold Calibration: Choosing the Operating Point

The J statistic is nearly flat across 0.70 to 0.74. Values in that band are statistically indistinguishable, which means the choice among them is a policy decision rather than a statistical one - a point worth making explicitly, because presenting any single value in that range as "the optimum" would overstate what the data support.

Threshold	Youden's J	Sensitivity	Specificity	Status
0.70	0.190	54.6%	64.4%	ROC point optimum
0.72	0.170	66.7%	50.3%	Adopted (bootstrap median)
0.75	0.169	86.7%	30.2%	Earlier recall-first choice

Table 5.2: Threshold operating points

The adopted value of 0.72 is the bootstrap median and sits in the middle of the plateau, balancing sensitivity against specificity. The earlier value of 0.75, which this project used for most of its duration, sat at the very upper edge of the confidence interval and traded specificity heavily for recall: it caught 86.7 per cent of improvable answers at the cost of needlessly regenerating roughly seven in ten answers that were already fine.

The bootstrap distribution is the strongest single piece of evidence that the answer is stable rather than an artefact of this particular corpus, and it is reproduced below.

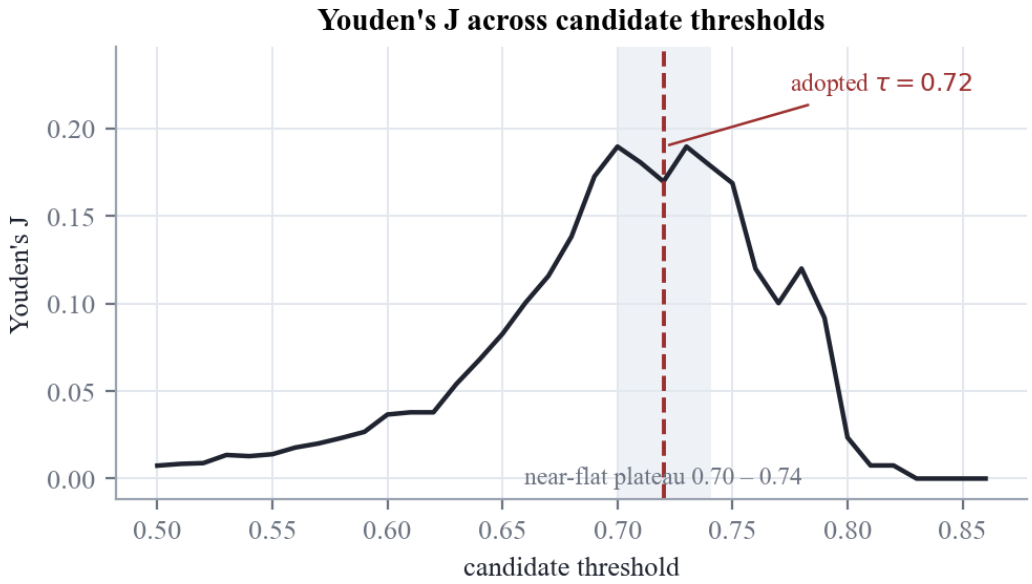


Figure 5.5: Youden's J across candidate thresholds, showing the near-flat plateau between 0.70 and 0.74

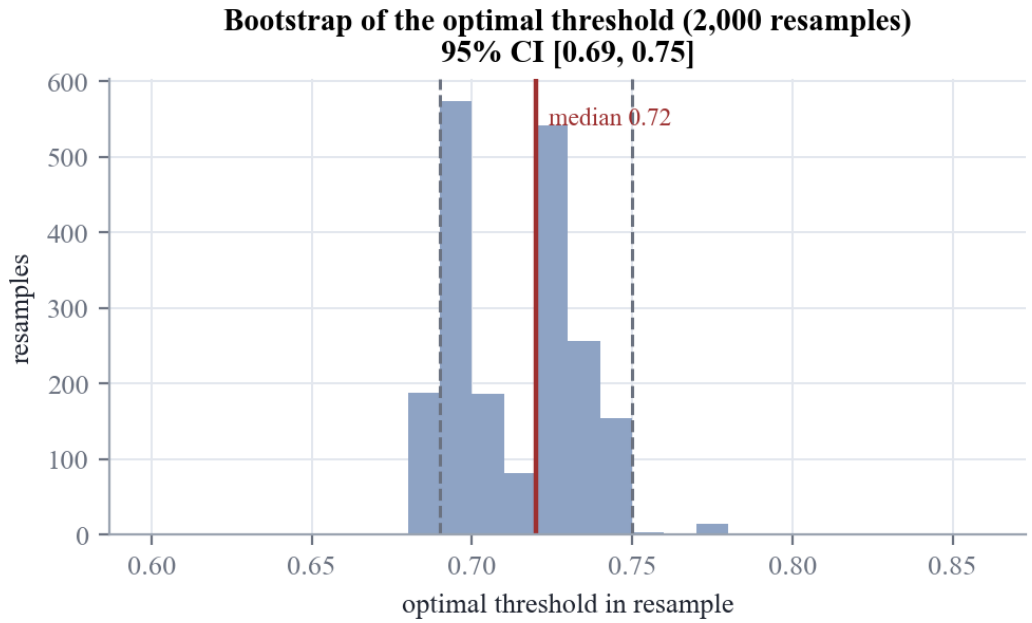


Figure 5.6: Bootstrap distribution of the optimal threshold over 2,000 weighted resamples

5.19 Evaluator Bias: Measuring the Combiner

The most substantial experimental result of this project concerns the Combiner, the agent that synthesises the final answer from the best attempt.

For most of the project the Combiner was llama3, which is also one of the two generators. It was therefore merging its own writing against another model's on every question. The symptom had already been recorded much earlier - similarity metrics computed against the final answer were biased toward one candidate - and the response at the time was to exclude

those metrics from the confidence score. That removed the bias from the measurements and left it entirely intact in the shipped answer.

The effect was measured across all 5,000 final records. Both candidate answers and the final answer were embedded with the same encoder used elsewhere in the pipeline, and the difference in cosine similarity between the final answer and each candidate gives a signed preference. Mapping that preference onto model identity using the recorded position mapping yields the identity lean; leaving it in slot terms yields the position lean.

Crossing the two factors is essential, because the marginal figures are misleading on their own. The headline number - 58.7 per cent of final answers closer to the llama3 candidate - conflates two distinct effects that happen to point the same way.

5.20 Evaluator Bias: Decomposition

Length was a credible competing explanation and was tested rather than dismissed. The llama3 generator does write longer answers on average - 366 words against Mistral's 317 - and a synthesiser that simply retained more of the longer source would produce exactly this pattern. However the correlation between length advantage and preference is only +0.029, and the effect persists at 61.1 per cent among length-matched pairs (word counts within 10 per cent, n = 1,645, $p = 2 \times 10^{-19}$). Length does not account for it.

The mechanism proved more specific than simple style recognition. The preference is conditional on the Judge agreeing with it. Where the Judge selected the llama3 answer, the final answer favoured it 69.4 per cent of the time. Where the Judge selected the other candidate, the figure was 50.7 per cent - not statistically distinguishable from chance ($p = 0.465$).

Stated as obedience: the Combiner followed the Judge 69.4 per cent of the time when the Judge pointed at the Combiner's own output, and only 49.3 per cent when it pointed elsewhere. A twenty-point asymmetry in how readily an instruction is followed, conditioned on whether the instruction favours oneself.

The full three-way crossing makes the structure visible, and every cell is individually significant. The decisive cell is the second: 1,815 records in which the llama3 answer occupied the first slot and the Judge had selected the other candidate. Mistral's answer was sitting there, selected, in the other slot - and the final answer still landed closer to llama3 in 64.0 per cent of cases.

Effect	Magnitude	Significance
Identity: own-answer preference, position held fixed	+18.1 pp	$p \approx 1.7 \times 10^{-89}$
Position: first-slot preference, identity averaged	+10.8 pp	$p \approx 5.7 \times 10^{-53}$
Length: correlation with word-count advantage	r = +0.029	Not supported

Table 5.3: Combiner bias, effects separated

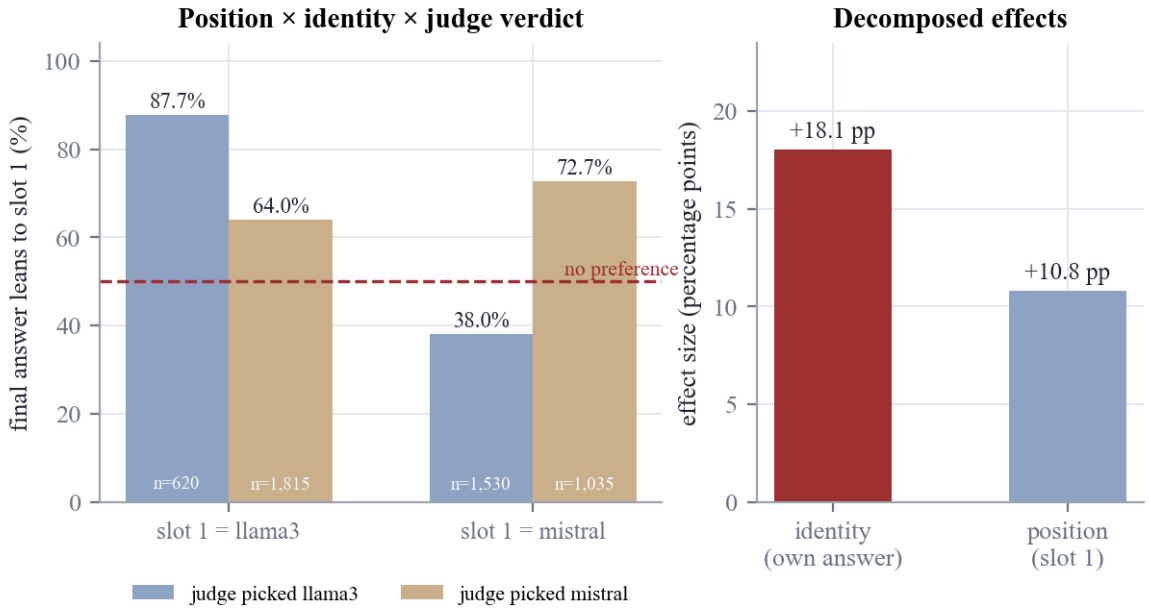


Figure 5.7: Combiner preference crossed by prompt position, model identity and judge verdict, with the two effects separated

5.21 Evaluator Bias: the Correction and its Limits

The Combiner was reassigned to gemma2:9b, a model that is neither generator, nor the Verifier, nor the Judge. The reasoning was that this removes SELF-preference by construction rather than reducing it: a model with no answer of its own among the two it is merging has nothing of its own to prefer. That reasoning is sound as far as it goes, but it does not follow that the replacement is therefore unbiased - a point the measurement below makes, and which would have been missed had the change been adopted on the argument alone.

A controlled replay was built to verify this rather than assume it. Because the Combiner is strictly terminal - the confidence score never takes the final answer as an input, and the Combiner runs once after the regeneration loop closes - every one of its inputs is already stored per record. The Combiner can therefore be re-run in isolation across the existing corpus, with the question, both candidates and both evaluator outputs frozen, and only the

model varied. The design crosses two models against two presentation orders, so that averaging the orders within a question cancels position exactly and isolates identity, while comparing them isolates position.

The replay required one precaution that is easy to overlook. When the presentation order is swapped, the "Answer 1" and "Answer 2" references inside the stored evaluator text must be remapped to match; without that, the Combiner receives a verdict pointing at the wrong candidate, which contaminates the very effect being measured.

The replay is now complete: 400 questions across two models and two presentation orders, 1,600 synthesis calls, with no failures. Its results are given in Table 5.8, and they are more nuanced than "the replacement worked".

Measure	llama3 (baseline)	gemma2:9b (adopted)
Prefers prompt slot 1	80.5%	50.7%
Leans toward the LLaMA 3 candidate	62.7%	35.2%
Distance from 50/50 parity on identity	12.7 pp	14.8 pp
Same, with answer length controlled	15.5 pp	7.3 pp
Mean output length (cap: 150 words)	223 words (max 543)	131 words (max 199)

Table 5.8: Controlled replay, 400 questions × 2 models × 2 orders

The position result is unambiguous and is the most important line in the table. Where the original Combiner tracked whichever answer it read first in 80.5 per cent of questions, the replacement does so in 50.7 per cent - within a single percentage point of perfect neutrality. This was not expected. The position effect had been described earlier in this chapter as a property of prompt reading rather than of model identity, and therefore as something a model swap could not address; the measurement contradicts that reasoning, and the reasoning was wrong rather than the measurement.

The identity result does not support the simpler claim that might have been made, and is reported as it stands. The replacement does not land on parity: it leans toward the Mistral candidate in 64.8 per cent of questions, which is 14.8 points from parity against the original's 12.7 points in the opposite direction. On the raw identity measure the replacement is marginally worse, and the lean has changed sign rather than disappeared.

That lean cannot be self-preference, because gemma2:9b has no answer of its own among the two it is merging. The obvious candidate explanation is length, and testing it accounts for about half the gap. The replacement compresses hard - 131 words on average against the original's 223 - so it must discard more, and the LLaMA 3 candidate is the longer source at 366 words against 317. Restricting to length-matched pairs moves the replacement from 35.2 per cent to 42.7 per cent, which is 7.3 points from parity; the same restriction moves the original the other way, from 62.7 to 65.5 per cent, confirming that length never explained the original's bias.

The honest summary is therefore three-part. The replacement eliminates the position effect, which was the larger structural defect. It eliminates self-preference by construction, since it has no self. It introduces a smaller, length-mediated lean toward the shorter source, which is a different and more tractable problem than an evaluator favouring its own output, and which an extractive selector would remove entirely. It also respects the 150-word instruction that the original exceeded by up to a factor of 3.6.

What remains unaddressed is that the per-question coin flip only ever made the position effect unbiased with respect to identity rather than absent - and for the live service, which fixed LLaMA 3 in slot one, not even that. Chapter 8 discusses extractive selection as the remedy that would remove both the residual position sensitivity and the length-mediated lean by construction.

5.22 Measurement Errors Detected and Corrected

Five errors were identified during the project that would have corrupted reported results had they gone unnoticed. They are recorded here because a report presenting only its successes misrepresents how the work was actually done, and because each was found by the same habit rather than by luck: a result that looked too clean was checked instead of accepted.

Error	How it was noticed	Consequence if unnoticed	Correction
Evaluator reported a rubric total inconsistent with its own criterion scores	A total of 55 against a stated maximum of 40	Inflated or deflated confidence scores	Totals recomputed from criterion scores
A 'second opinion' signal was derived from the Judge's own output	It correlated with the Judge too tightly to be independent	Self-agreement silently inflating confidence	Signal removed from the score

Error	How it was noticed	Consequence if unnoticed	Correction
Threshold analysis applied one sampling weight although the control rate changed mid-run	ROC gave AUC ≈ 0.51 and an optimum below the current threshold, contradicting a clean earlier result	A mis-weighted, wrong threshold	Per-record weighting by the rate actually in force
Restart-orphaned attempts produced duplicate final rows for four questions	5,003 final rows for 5,000 questions	Aggregate statistics over a corpus of the wrong size	Rows corrected; the write now targets a unique row id
The results viewer labelled winners by prompt slot, not model identity	A screenshot showing 23 of 24 recent wins to one model	A false conclusion that the pipeline was overwhelmingly biased	Winner name derived from the recorded identity columns

Table 5.4: Measurement errors and corrections

The third, fourth and fifth deserve comment, because each produces plausible output rather than an error.

The third arose because the control probability was raised from 0.08 to 0.60 partway through the run. Applying the current value to every control record undercounts those sampled under the old rate by a factor of 7.5, which corrupts sensitivity, specificity and AUC - all of which depend on population composition - while leaving the simple per-band rates untouched. The symptom was that the ROC analysis contradicted a band comparison that had already been established as clean and significant. Two analyses of the same data disagreeing is informative; the disagreement was traced to the weighting rather than averaged away.

The fourth is a schema trap. Attempts were being marked final by matching on question index and attempt number, a pair that appears unique but is not. If the process is interrupted between inserting an attempt and marking it final, the question is reprocessed from scratch on restart, and its fresh first attempt shares that pair with the orphaned one - so both get flagged. Four questions were affected. The fix was to return the row identifier at insertion and target the update by that identifier alone.

The fifth never touched the data at all, which is what makes it worth recording. The results viewer rendered the winning model's name as "A means LLaMA 3, B means Mistral" - true before answer order was randomised, and never updated afterwards. The stored data was correct throughout; only the display was wrong. It surfaced when a screenshot showed 23 of 24 recent questions won by one model, which looked like overwhelming bias. Checking those exact rows in the database showed the opposite: the coin flip had simply landed the

same way for a stretch, and the true split was close. A reader of that screen would have drawn a firm and entirely false conclusion about the system, from data that was never wrong.

5.23 The Retrieval Path Was Inert Throughout

Section 3.10 describes a retrieval subsystem with a minimum-similarity gate, and Section 3.4 defines a Context Relevance criterion scored against retrieved material. Both descriptions are accurate about the implementation and misleading about the corpus, and the discrepancy was found late.

The batch harness calls its per-attempt routine with the retrieval-context parameter left at its default of empty, and never overrides it. Every one of the 5,000 questions in the calibration corpus therefore ran with retrieval completely inert. No chunk was ever retrieved, no similarity gate was ever evaluated, and the Context Relevance criterion was scored by its fallback definition - whether the answer stays on topic - on every single record.

Three consequences follow, and all three limit what this report can claim.

First, the minimum-similarity threshold of 0.35 has never been calibrated and, unlike the confidence threshold, currently cannot be. This is not a sample-size problem that more generation would solve; it is the absence of a data path. The parameter only executes in the live interactive service, which sees a tiny fraction of the batch harness's traffic, and until recently that service recorded no retrieval outcome anywhere persistent. There is nothing in the corpus against which to fit it.

Second, the Context Relevance scores reported in Table 5.5 - Judge 8.38, Verifier 9.25 - measure topic adherence, not fidelity to supplied material. They should not be read as evidence that the system grounds answers in documents, because on this corpus it never had any.

Third, the retrieval subsystem as a whole is untested at scale. It works in the live service and was exercised during development, but no claim in Chapter 5 rests on it, and none should be read as doing so.

Instrumentation has since been added: the live service now logs, per query, whether context was injected, how many chunks, and the similarity of the best match. That provides the data path the calibration needs, by the same control-group method used for the confidence threshold - injecting context on a random fraction of queries that fall below the gate, to

produce the observations the gate otherwise suppresses. The calibration itself remains outstanding and is listed in Chapter 8.

This is reported in the results chapter rather than buried in limitations because it bears directly on how two of the tables above should be read.

5.24 Threats to Validity

Several threats bound the conclusions of this chapter and are stated explicitly.

Construct validity. The label-free improvement criterion defines "better" as an increase across the five non-judge signals. This is a proxy for quality, not quality itself, and it was chosen specifically to exclude the Judge and avoid circularity. A system that improved on those five signals while becoming less accurate would be scored as improved, and this methodology would not detect it.

Internal validity. The AUC of 0.629 indicates weak discrimination. The threshold is calibrated on a real relationship, demonstrated significant by an independent test, but it is not a strong one, and the near-flat J curve reflects that. A stronger predictor would produce a sharper optimum.

External validity. All results are specific to this model set, this corpus and this rubric. The protocol is expected to transfer and the numbers are not; this has not been tested.

Statistical conclusion validity. The very small p-values reported for the bias effects reflect a large corpus and should be read as evidence that the effects are real, not that they are large. The effect sizes - 18.1 and 10.8 percentage points - are the quantities of practical interest, and they are reported alongside every p-value for that reason.

Instrumentation validity. The retrieval path was inert for the entire corpus (Section 5.14), so the Context Relevance scores measure topic adherence rather than fidelity to supplied material, and no claim here rests on the retrieval subsystem. Separately, two of the five errors in Table 5.4 were in the measurement and display layers rather than in the pipeline, which is a reminder that a correct system observed through a faulty instrument is indistinguishable from a faulty system.

Sample size in the replay. The controlled replay of Section 5.9 used 400 of the 5,000 questions. At that size the 95 per cent interval on a proportion is roughly five percentage points, which is comfortably narrower than the effects reported - a 30-point shift in position preference and a 27-point shift in identity lean - but not narrow enough to support fine

distinctions. The claim that the replacement sits 7.3 points from parity when length is controlled should be read as "closer to parity than the baseline", not as a precise figure.

Unresolved disagreement. Section 5.8 reports two evaluators disagreeing about whether the generators differ at all, and the data do not separate the two readings. This is the largest single open question in the work, and it propagates: if the Judge's preference is an artefact rather than a quality signal, then the winner column - and therefore the Judge's contribution to every confidence score - carries that artefact. Nothing in this report rules that out.

Chapter 6 System Testing and Validation

6.1 Testing Strategy

The system has two kinds of correctness and they require different evidence. Functional correctness asks whether the implementation does what it was specified to do: whether the gate fires at the right score, whether a missing signal is renormalised rather than treated as zero, whether a restart resumes where it left off. Behavioural correctness asks whether the components exhibit the properties the architecture assumes: whether an evaluator can actually discriminate, whether randomisation is balanced, whether regeneration improves anything.

Chapter 5 addresses the second kind. This chapter addresses the first, together with the robustness testing that an unattended multi-day run required.

No automated unit test suite was maintained for the agent layer, and that is a limitation rather than a decision to defend. Agent outputs are non-deterministic text, so assertions about them are either trivially weak or brittle. Testing concentrated instead on the deterministic parts - score computation, parsing, persistence, resumption - and on observing the non-deterministic parts through the streaming interface.

6.2 Score Computation and Renormalisation

The weighted score function was exercised against hand-computed cases, because an error here would silently corrupt every downstream result and would not announce itself.

The cases checked were: all six signals present, which must reproduce the plain weighted sum; a single signal present, which must return that signal's own value because its renormalised weight is one; each external signal absent individually, which must redistribute its weight proportionally rather than treating the absence as a zero score; and no signals at all, which must raise rather than return a misleading zero.

The distinction between an absent signal and a zero-valued signal is the one that matters. An absent Wikipedia check means "this claim could not be verified"; a Wikipedia score of zero means "this claim was checked and contradicted". Collapsing the two would systematically penalise questions about topics with no encyclopedia coverage, and given the 44 per cent availability reported in Section 5.6, that error would have affected the majority of records.

6.3 Rubric Parsing

Evaluator output is free text in a requested format, and models do not always honour the format. The parser was therefore tested against the failure modes actually observed in the corpus rather than against an idealised specification.

Observed and handled: a criterion omitted entirely; a score written as a bare integer rather than as a fraction; a stated total inconsistent with the criterion scores; an extra "## Answer 1" heading appearing mid-output, which occurs in 625 of the 8,432 verifier outputs; and a reversal in which the model scores answer 2 before answer 1.

The handling policy is to recompute rather than to trust. Totals are summed from criterion scores. A missing criterion yields a missing value which propagates as an absent signal rather than a zero. Where the output cannot be parsed at all, the attempt is recorded with null scores rather than discarded, so that the failure is visible in the corpus rather than silently reducing the sample.

6.4 Gate and Regeneration Logic

The regeneration gate was verified against three properties.

First, the boundary: an attempt scoring exactly the threshold passes, and one scoring fractionally below it does not. The comparison is strictly less-than, so 0.72 passes and 0.7199 does not.

Second, the attempt bound: no question may produce more than three attempts. The corpus confirms this empirically - 5,005 first attempts, 2,001 second and 1,426 third, with no fourth.

Third, and most importantly, monotonicity of the shipped answer. The highest-scoring attempt is retained even when a later attempt scores lower, so regeneration cannot degrade the output. This was verified by querying the corpus for any question whose final flagged attempt was not its highest-scoring attempt; the query returns nothing.

6.5 Randomisation Balance

Position assignment is a per-question coin flip, and a biased or broken randomiser would invalidate every bias analysis in Chapter 5. Over the 5,000 final records, LLaMA 3 occupied the first slot in 2,435 cases and Mistral in 2,565 - 48.7 per cent against 51.3 per cent.

For a fair coin over 5,000 trials the standard deviation of the count is approximately 35, so the observed deviation of 65 from the expected 2,500 is under two standard deviations and

entirely consistent with a fair process. The randomisation is sound, which is what licenses treating the position and identity factors as independent in Section 5.8.

6.6 Persistence and Restart Behaviour

The harness must resume correctly after interruption, because it was interrupted repeatedly in practice. Resumption was tested by terminating the process at several points in the attempt cycle and restarting it: between attempts, between the final attempt and the combiner call, and during the database write.

The third case is the one that exposed a real defect. Terminating between inserting an attempt and marking it final leaves an orphaned row, and on restart the question is reprocessed from scratch and produces a fresh attempt sharing the same question index and attempt number. The original code identified the row to flag by that pair, so both rows were flagged. Four questions in the corpus were affected, yielding 5,003 final rows for 5,000 questions.

This is reported in Section 5.10 as a measurement error, and it is reported here as a test outcome, because it is both: a bug that corrupted a statistic, and a defect that only a deliberate mid-write termination test would have surfaced. The fix returns the row identifier at insertion and targets the update by that identifier alone.

6.7 Failure Modes and Degradation

The system's behaviour was characterised under several failure conditions rather than assumed.

Condition	Observed behaviour	Handling
Model runtime unreachable	Agent call raises	Health endpoint reports unavailable models before a run starts
Evaluator returns empty text	Parse yields no scores	Attempt recorded with null scores; signal absent, not zero
Reasoning-enabled judge on a long prompt	Empty response, no error	Reasoning disabled in configuration
External lookup finds no entity	No score returned	Weight redistributed across remaining signals
Two large models requested concurrently	VRAM thrash, throughput collapse	Concurrency fixed at one; evaluators sequential
Process terminated mid-write	Orphaned attempt row	Final flag targeted by unique row id
Host sleeps or session closes	Generation silently stops	External scheduled watchdog restarts the process

Table 6.1: Observed failure modes and system response

The second row is worth emphasising because it is the dangerous one. An evaluator returning empty text produces no error anywhere in the stack: the HTTP call succeeds, the string is well-formed, and the parser simply finds nothing in it. Without an explicit check this failure presents as a quiet reduction in signal availability rather than as a fault, and it would be easy to attribute to the external sources instead.

6.8 End-to-End Verification

After each significant change the full pipeline was exercised against a live question through the public interface, and the event stream inspected for all six stage-completion events, a non-null final answer, and a plausible confidence score.

The most recent such run, following the Combiner change of Section 5.8, completed all six stages and produced a 136-word final answer - within the 150-word instruction that the previous Combiner had been exceeding by up to a factor of 2.7.

One observation from these runs is worth recording for anyone reproducing the work. Under GPU contention - when an analysis job is running concurrently - the same query took 327 seconds rather than the usual 125, because the runtime spends most of its time evicting and reloading model weights rather than generating. The system does not fail under contention; it degrades, substantially and silently. On a single-GPU host, the pipeline and any analysis workload should not be run at the same time.

Chapter 7 Conclusion and Recommendations

7.1 Conclusions

This project set out to determine whether a locally hosted question-answering system can be validated without human-annotated ground truth, and the answer is a qualified yes. Internal consistency, evaluator discrimination, relative improvement under regeneration and the calibration of an operating threshold are all achievable label-free. Absolute factual accuracy is not, and no such claim is made anywhere in this report.

The threshold calibration demonstrates that a parameter commonly chosen by intuition can be derived from data even when those data are censored by the parameter itself, provided a control group is deliberately introduced to produce the observations the system would otherwise never generate. Three independent statistical routes converged on the same narrow band, and the adopted value of 0.72 is the bootstrap median of that band. The discrimination underlying it is weak - an AUC of 0.629 - and that is reported rather than obscured.

The most significant finding is methodological. Turning the measurement apparatus on itself revealed that the synthesising agent preferred its own generated answers by 18.1 percentage points, with a separate 10.8 point preference for the first prompt slot, and that neither effect is explained by answer length. This was not a suspected weakness confirmed; it was a quantified defect in the project's own prior design, found by measurement, and corrected. The mechanism - selective obedience to an evaluator depending on whether the evaluator's verdict was flattering - is more specific and more interesting than the generic "self-preference" the literature describes.

A system of this kind is therefore best understood not as a correctness oracle but as a calibrated quality gate. It reliably separates weaker answers from stronger ones and acts on that separation, while remaining honest about what it cannot certify.

7.2 Recommendations

- No model should both generate and evaluate within an evaluation pipeline. The cost of violating this is measurable, and was measured here at 18.1 percentage points.
- Randomising presentation order is necessary but not sufficient. It makes a position effect unbiased with respect to identity; it does not remove the effect. Systems should

state which of the two they have achieved rather than claiming to have addressed position bias.

- Any threshold governing a feedback loop should be calibrated against a deliberately injected control group, because the loop censors exactly the observations needed to calibrate it.
- Evaluator arithmetic should be recomputed rather than trusted, and any signal that performs unexpectedly well should be traced to its source before being relied upon.
- Effect sizes should be reported alongside p-values. On a corpus of several thousand records, significance is cheap and magnitude is the quantity of interest.
- Measurement errors should be documented rather than quietly corrected. Several of the most useful findings in this project originated in investigating an anomaly that could have been smoothed over.

Chapter 8 Future Scope

8.1 Eliminating the Position Effect by Construction

The residual position effect arises because a generative synthesiser reads a prompt in order and over-weights what it reads first. An extractive approach would not have this property. Maximal marginal relevance selects sentences by a combination of relevance to the query and non-redundancy with what has already been selected; it scores content, and prompt position is not an input to the scoring at all.

Such an approach would remove both the position and identity terms by construction. It would also make the contribution of each source directly countable rather than inferred from embedding similarity - one could simply report what fraction of the selected sentences came from each generator, turning bias from an emergent property into a design parameter that can be constrained. A useful side effect is that it would calibrate the embedding-based lean metric used throughout Section 5.8, since the true source split would be known exactly.

One property of maximal marginal relevance must be addressed before adopting it. Its diversity term rewards selecting sentences dissimilar to those already chosen, and a contradiction is maximally dissimilar. The method is therefore structurally inclined to include both sides of a factual disagreement, which is the opposite of what a synthesiser should do and directly conflicts with the Combiner's stated job of removing contradictions the Verifier flagged.

A hybrid design addresses this. Extraction selects the content - bias-free, balanced, length-exact and auditable - and a small model performs only surface realisation, smoothing transitions and resolving dangling references, explicitly forbidden from adding or removing material. The selection decision, which is where the bias lived, moves out of the language model entirely, while fluency and contradiction handling are retained.

8.2 A Human-Annotated Validation Study

The label-free methodology establishes internal properties but cannot establish absolute accuracy. A blind, balanced study in which human raters score a stratified sample of answers would allow the confidence score to be validated against human judgement, and would convert the system's relative claims into absolute ones.

A rating harness and a pre-registered analysis for such a study have been prepared: a balanced sample, a blind presentation order, and a binomial test specified in advance so

that the analysis cannot be chosen after seeing the data. What remains is the rating effort itself. Even a modest study of a few hundred items would materially strengthen every quality claim in Chapter 5.

8.3 Calibrating the Retrieval Threshold

The retrieval subsystem applies a minimum similarity threshold below which retrieved material is excluded, so that an unrelated document cannot contaminate an answer. That threshold has not received the same treatment as the confidence threshold; it was set by inspection.

Retrieval decisions are now logged per query - whether context was injected, how many chunks, and at what similarity - which provides exactly the data required to calibrate it by the same control-group method. The analogous control would inject context on a random fraction of queries that fall below the threshold, producing the censored observations the gate otherwise suppresses.

8.4 Distillation of the Judge

Evaluation latency is dominated by the Judge, a nine-billion-parameter model invoked at least once per attempt. Training a small regression model to replicate the Judge's per-criterion rubric scores from the question-answer pair would reduce evaluation to milliseconds and permit the pipeline to run at interactive speed.

This direction was explored early in the project and set aside in favour of establishing the validation methodology first - a sequencing decision that proved correct, since the bias findings of Section 5.8 would have been distilled into the student model had it been trained on the uncorrected pipeline. The corpus generated by this project is well suited to the task, containing per-criterion scores from two independent evaluators for 8,432 attempts.

The obvious caution is that a distilled judge inherits whatever biases the teacher had. Distillation should follow bias correction, not precede it.

8.5 Broader Signal Set and Adaptive Weighting

The six signals and their weights were chosen as a reasoned prior and have been only partially swept. A systematic weight optimisation against the control-group improvement criterion would establish whether the current allocation is near-optimal or merely reasonable.

Further signals are available without additional model calls: answer-length distribution relative to question type, named-entity overlap between the two candidates, and the variance of per-criterion scores within a single evaluator, which may itself indicate uncertainty. Whether any of these adds discriminative power beyond the existing six is an empirical question the corpus can answer.

8.6 Generalisation

The protocol developed here - blind evaluation, discrimination testing, control-group calibration under censoring, and bias decomposition with confounds crossed - is model-agnostic and domain-agnostic. The tuned values are not.

Re-deriving them on a different model set would test whether the method transfers while the numbers do not, which is the expected outcome and is stated here as a hypothesis rather than a result. A particularly informative variant would substitute two generators from the same family, where correlated errors should measurably weaken the cross-model agreement signal - a prediction the methodology makes and that could be falsified.

References

[1] L. Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[2] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu and C. Zhu, “G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment,” in Proc. EMNLP, 2023.

[3] S. Kim et al., “Prometheus: Inducing Fine-grained Evaluation Capability in Language Models,” in Proc. ICLR, 2024.

[4] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger and Y. Artzi, “BERTScore: Evaluating Text Generation with BERT,” in Proc. ICLR, 2020.

[5] K. Papineni, S. Roukos, T. Ward and W. Zhu, “BLEU: a Method for Automatic Evaluation of Machine Translation,” in Proc. ACL, 2002.

[6] C.-Y. Lin, “ROUGE: A Package for Automatic Evaluation of Summaries,” in Text Summarization Branches Out, ACL Workshop, 2004.

[7] Q. Wu et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” arXiv:2308.08155, 2023.

[8] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[9] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proc. EMNLP, 2019.

[10] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in Proc. NAACL-HLT, 2019.

[11] W. J. Youden, “Index for rating diagnostic tests,” Cancer, vol. 3, no. 1, pp. 32–35, 1950.

[12] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap. New York: Chapman & Hall, 1993.

[13] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” Journal of the American Statistical Association, vol. 22, no. 158, pp. 209–212, 1927.

[14] J. A. Hanley and B. J. McNeil, “The meaning and use of the area under a receiver operating characteristic (ROC) curve,” Radiology, vol. 143, no. 1, pp. 29–36, 1982.

[15] J. Carbonell and J. Goldstein, “The use of MMR, diversity-based reranking for reordering documents and producing summaries,” in Proc. ACM SIGIR, 1998.

[16] AI@Meta, “Llama 3 Model Card,” 2024.

[17] A. Q. Jiang et al., “Mistral 7B,” arXiv:2310.06825, 2023.

[18] M. Abdin et al., “Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone,” arXiv:2404.14219, 2024.

[19] Gemma Team, “Gemma 2: Improving Open Language Models at a Practical Size,” arXiv:2408.00118, 2024.

[20] A. Yang et al., “Qwen2.5 Technical Report,” arXiv:2412.15115, 2024.

[21] D. Vrandečić and M. Krötzsch, “Wikidata: a free collaborative knowledgebase,” Communications of the ACM, vol. 57, no. 10, pp. 78–85, 2014.

[22] A. Agresti and B. A. Coull, “Approximate is better than exact for interval estimation of binomial proportions,” The American Statistician, vol. 52, no. 2, pp. 119–126, 1998.

Appendices

Appendix A: Agent Configuration

Each agent is instantiated with a role description, a model identifier, a decoding temperature and a token budget, as given in Table 3.1. The table is reproduced here with the rationale for each decoding choice.

Generators run at temperature 0.7. Diversity between the two candidates is the point of having two, and a lower temperature would produce near-identical answers from which no agreement signal could be derived.

Evaluators run at temperature 0.1. Scoring should be as close to deterministic as the runtime permits; an evaluator returning different scores for identical input on consecutive calls cannot support any claim about the system.

The Combiner runs at 0.4, between the two. Synthesis requires enough freedom to produce fluent connective prose, and not enough to invent material absent from both sources.

The Judge additionally sets the reasoning flag to false. With internal reasoning enabled, qwen3.5:9b expends its entire generation budget on deliberation and returns an empty string on long rubric prompts - a silent failure, since the caller receives a well-formed but empty response rather than an error.

Appendix B: Evaluator Prompt Structure

Evaluator prompts are brand-blind. They present the question and both candidate answers, labelled only as "Answer 1" and "Answer 2", with no reference to which system produced either. The Verifier prompt has the following structure, with the retrieved-context block present only when retrieval is active:

An optional block of retrieved reference documents; then the question; then the two answers under "--- ANSWER 1 ---" and "--- ANSWER 2 ---" headings; then the instruction to score both answers on all eight criteria, writing one sentence per score explaining why and referencing specific content from the answer. Where no reference documents were supplied, Context Relevance is to be scored on whether the answer stays on topic. The prompt closes by requiring the total to be computed as the sum of the eight scores, to a maximum of eighty.

The Judge prompt follows the same structure and additionally requires an overall winner and a short reasoning paragraph. The Judge does not receive the Verifier's scores; the two evaluations are independent, which is what makes their agreement a usable signal rather than an echo.

One property of the evaluator output matters downstream and is easy to overlook. The reasoning prose refers to the candidates by label - "Answer 1 adheres more strictly to the prompt", for instance. Any downstream consumer that reorders the answers must remap those labels, or it will act on a verdict pointing at the wrong text. Section 5.10 records that exactly this defect was present in the live service.

Appendix C: The Eight-Factor Rubric in Full

Each criterion is scored from 0 to 10 for each answer, for a maximum of 80 per answer, and each score requires a one-sentence justification.

Factual Correctness: whether the facts stated are accurate. Catches confident but incorrect dates, figures and names.

Question Relevance: whether the answer addresses the question actually asked. Catches answers that drift to an adjacent, easier question.

Context Relevance: consistency with supplied reference material, or, where none was supplied, whether the answer remains on topic.

Faithfulness: whether the claims made are supportable rather than invented. Catches plausible but unsupported specifics.

Completeness: whether the answer covers the question adequately. Catches answers addressing only one half of a two-part question.

Hallucination-Free: absence of fabricated specifics such as invented citations, statistics or quotations.

Clarity: structure and readability. A correct answer the reader cannot follow is of limited use.

Consistency: absence of internal self-contradiction within the answer.

Totals are recomputed from the criterion scores rather than read from the model's stated total, because evaluators were observed reporting totals inconsistent with their own scores.

Appendix D: Confidence Score Computation

For the set S of signals available for a given answer, with values v normalised to the range [0, 1] and nominal weights w from Table 3.3, the weighted confidence score is the sum over S of v multiplied by w, divided by the sum over S of w.

The division renormalises the weights across whichever signals are present, so the score remains on the same scale regardless of how many contributed. The Judge's rubric total is normalised by division by eighty; the remaining signals are natively in range.

A signal is unavailable when its source produced nothing - most commonly when an external encyclopedia check found no sufficiently relevant article or entity. Returning nothing rather than a low score is deliberate: it avoids conflating "unverifiable" with "wrong", which would penalise questions about obscure topics for the absence of an encyclopedia entry.

Deliberately excluded are any signals referenced against the final combined answer. Such signals were found to be biased toward whichever candidate the Combiner had been built from, for the reasons quantified in Section 5.8.

Appendix E: Database Schema

One row is written per attempt into the pipeline_runs table. The principal columns are as follows.

Identity and sequencing: id (primary key), question_idx, question, attempt_number, is_final_attempt.

Candidates and provenance: answer_a, answer_b, model_a, model_b - the last two recording which physical model occupied which prompt slot for this question, which is what makes the identity analyses in Section 5.8 possible after the fact.

Raw evaluator output: verifier_raw, judge_raw - the complete text returned by each evaluator, retained so that an analysis months later can reconstruct exactly what each evaluator saw.

Parsed scores: for each of the eight criteria, four columns - verifier and judge, answer A and answer B - plus verifier_total_a, verifier_total_b, judge_total_a, judge_total_b and winner.

Signals: sem_sim_a, sem_sim_b, length_score_a, length_score_b, word_count_a, word_count_b, model_agreement, verifier_judge_agreement, wikipedia_score, wikipedia_source, wikidata_score, wikidata_source.

Outcome: weighted_score, regen_reason, final_answer, elapsed_secs, created_at.

Indexes on question_idx and is_final_attempt were added after a paginated browse query over the full corpus began timing out at high offsets; with them the worst-case page load returns in under a fifth of a second.

Appendix F: Threshold Calibration Procedure

The calibration proceeds in five steps.

First, a control group is formed: a random fraction of answers passing the gate are regenerated regardless, producing observations the gate would otherwise censor.

Second, each record is weighted by the inverse of the control probability actually in force when it was generated. That probability changed from 0.08 to 0.60 partway through the run, so records carry weight 12.5 or approximately 1.667 accordingly, while census records - those regenerated because they genuinely failed the gate - carry weight 1.0.

Third, an improvement label is derived for each record from the five non-judge signals. The Judge is deliberately excluded from the improvement criterion to avoid circularity, since the Judge's own score is the dominant input to the threshold being calibrated.

Fourth, the receiver-operating-characteristic curve is computed over a grid of candidate cutoffs from 0.50 to 0.86 in steps of 0.01, and Youden's J maximised.

Fifth, the whole procedure is repeated over 2,000 bootstrap resamples of the labelled question set, with weights preserved, to obtain a confidence interval for the location of the optimum.

The analysis is reproducible by running training/optimal_threshold_roc.py. The figures in Section 5.7 are produced by make_figures.py, which imports that script's own functions and uses its grid and random seed, so that the plates cannot drift from the numbers in the text.

Appendix G: Combiner Bias Measurement Procedure

For each final record, the two candidate answers and the final answer are embedded with all-MiniLM-L6-v2, the same encoder used elsewhere in the pipeline. The difference in cosine similarity between the final answer and each candidate gives a signed preference for that record.

Mapping that preference onto model identity using the recorded model_a and model_b columns yields the identity lean; leaving it in slot terms yields the position lean. Crossing the two, and further crossing by the Judge's recorded verdict, separates effects that are conflated in the marginal figures - which is necessary here, because the two effects point the same way and the marginal number is therefore misleading on its own.

Length is controlled in two ways: as a correlation between the word-count advantage and the preference, and by restricting the analysis to pairs whose word counts fall within ten per cent of one another.

The controlled replay freezes the question, both candidates and both evaluator outputs, and varies only the Combiner model and the presentation order. When the order is swapped, the "Answer 1" and "Answer 2" references inside the evaluator text are remapped accordingly; without that remapping the Combiner would receive a verdict pointing at the wrong candidate, contaminating the measurement.

The analysis is reproducible by running training/combiner_bias_analysis.py, and the replay by training/combiner_replay.py.

Appendix H: Full ROC Table

The complete sampling-corrected receiver-operating-characteristic table underlying Section 5.7 and Figure 5.4. At each candidate cutoff t, an answer is "flagged" if its attempt-1 confidence score falls below t. Sensitivity is the weighted fraction of truly improvable answers correctly flagged; specificity is the weighted fraction of truly satisfactory answers correctly left alone. Youden's J is their sum less one. Area under the curve: 0.6291.

t	n flagged	n ok	Sensitivity	Specificity	Youden J
0.50	31	1973	1.2%	99.5%	0.0074
0.55	71	1933	2.7%	98.7%	0.0139
0.60	199	1805	7.3%	96.3%	0.0366
0.63	362	1642	12.7%	92.7%	0.0541
0.64	452	1552	15.9%	90.9%	0.0679
0.65	565	1439	19.8%	88.5%	0.0827
0.66	720	1284	25.0%	85.1%	0.1005
0.67	896	1108	30.6%	80.9%	0.1158
0.68	1128	876	38.2%	75.6%	0.1383
0.69	1245	759	47.7%	69.6%	0.1727
0.70	1360	644	54.6%	64.4%	0.1897
0.71	1464	540	59.7%	58.4%	0.1807
0.72	1593	411	66.7%	50.3%	0.1697
0.73	1707	297	75.0%	43.9%	0.1897
0.74	1825	179	81.7%	36.2%	0.1792
0.75	1915	89	86.7%	30.2%	0.1688
0.76	1937	67	89.7%	22.3%	0.1199
0.78	1968	36	97.3%	14.7%	0.1200

t	n flagged	n ok	Sensitivity	Specificity	Youden J
0.80	2000	4	100.0%	2.3%	0.0235
0.83	2004	0	100.0%	0.0%	0.0000

Table H.1: Sampling-corrected ROC across all candidate cutoffs

Two features of the table deserve comment. The maximum of J is attained twice, at 0.70 and again at 0.73, both at 0.1897 - which is the flatness described in Section 5.7 made explicit rather than asserted. And above 0.80 the specificity collapses toward zero while sensitivity saturates at 100 per cent: at those cutoffs essentially every answer is flagged, the gate stops discriminating, and the regeneration loop would run on almost every question to no purpose.

Appendix I: A Worked Example

One complete record from the corpus, traced through the pipeline. Question index 4: "How does a neural network learn from data?"

Position assignment for this question put Mistral in slot A and LLaMA 3 in slot B. The Judge declared answer B the winner, with rubric totals of 70 for A and 75 for B; the Verifier, scoring independently, reached the opposite ordering at 75 for A and 68 for B. This disagreement is exactly the situation the Verifier-Judge agreement signal exists to detect, and it scored 0.925 - high, because the agreement measure compares the evaluators' assessment of the winning answer rather than requiring them to pick the same winner.

Signal	Value	Weight	Contribution
Judge rubric total (B)	75 / 80 = 0.938	0.40	0.375
Semantic similarity (B)	0.650	0.15	0.097
Wikipedia check	0.438	0.15	0.066
Model agreement (BERTScore A–B)	0.098	0.10	0.010
Verifier–Judge agreement	0.925	0.10	0.093
Wikidata check	0.404	0.10	0.040
Weighted confidence score	—	1.00	0.681

Table I.1: Signals for question 4, attempt 2

The winning answer's per-criterion scores from the Judge were 9, 9, 9, 10, 9, 10, 10, 9 across the eight criteria in rubric order, totalling 75. The Verifier scored the same answer 8, 8, 8, 9, 8, 9, 9, 9, totalling 68 - the systematic leniency gap of Section 5.5 running in the opposite direction here, which is a reminder that it is a tendency across 5,000 records rather than a rule holding on each.

The attempt history shows the gate working and then exhausting itself. Attempt 1 scored 0.6744, below the threshold, and triggered a regeneration. Attempt 2 scored 0.6808 - an improvement, but still below 0.72. The attempt bound was reached and the better of the two was shipped. This record is therefore one of the 1,849

final answers, 37.0 per cent of the corpus, that ship below the confidence threshold because the pipeline could not do better within its attempt budget.

Two further details are visible in this record. Model agreement is 0.098, very low: the two generators produced substantively different explanations of the same concept, which is the expected behaviour described in Section 5.6 and the reason that signal contributes only 0.010 to the final score here. And the shipped answer runs to 285 words against a prompt instruction of at most 150 - this record predates the Combiner change of Section 5.8, and the instruction-following failure it illustrates was one of the secondary reasons for that change.

Appendix J: Verbatim Evaluator Output

A complete, unedited Judge output from the corpus, for the question "What is epigenetics?". It is reproduced to show the structure the parser consumes and the quality of justification the rubric elicits. Only the first answer's block is shown; the second follows the same form, after which the Judge declares a winner and gives an overall reasoning paragraph.

Answer 1

- Factual Correctness: 9/10 | The definitions of epigenetics and the four listed mechanisms (DNA methylation, histone modification, chromatin remodeling, non-coding RNA) are scientifically accurate and standard.

- Question Relevance: 10/10 | Every sentence directly defines or elaborates on epigenetics without including unrelated tangents or generic introductions.

- Context Relevance: 9/10 | The answer aligns perfectly with general scientific consensus on epigenetics since no specific external context was provided to contradict.

- Faithfulness: 9/10 | All claims regarding mechanisms and biological roles are grounded in established biological knowledge without fabrication.

- Completeness: 9/10 | The response covers the definition, causes, specific mechanisms, and key biological implications comprehensively.

- Hallucination-Free: 10/10 | No invented statistics, names, or false causal links were introduced; all examples are standard textbook instances.

- Clarity: 9/10 | The use of numbered lists for mechanisms and processes makes the complex information easy to scan and understand.

- Consistency: 9/10 | No internal contradictions appear between the definition given and the mechanisms subsequently described.

Three properties of this output are worth drawing out. The justifications reference specific content from the answer rather than restating the criterion, which is what the prompt asks for and what distinguishes a usable score from a reflexive one. The scores are differentiated across criteria rather than uniform, which is the property that disqualified a candidate judge in Section 5.3. And the Context Relevance justification explicitly notes that no external context was supplied and states how it scored in that case - the fallback behaviour defined in the rubric, correctly applied.

The output is also the reason totals are recomputed rather than trusted. The eight scores above sum to 74; an evaluator that states its own total introduces an opportunity for arithmetic that does not match, and such mismatches were observed.

Appendix K: Glossary of Terms as Used in This Report

Several terms in this report carry a narrower meaning than their general usage, and a few are used in a sense that differs from a common one. They are defined here as used.

Term	Meaning as used here
Attempt	One pass through stages 1-5 for a question. A question may have up to three.
Final answer	The output of the Combiner for the highest-scoring attempt of a question.
Candidate	One of the two independently generated answers, before synthesis.
Confidence score	The weighted blend of six signals, in [0,1]. A ranked quality signal, NOT a probability of correctness.
Calibrated	Of the threshold: chosen by diagnostic-test methodology. NOT calibration in the probabilistic sense (see Section 2.9).
Label-free	Requiring no human-annotated ground truth at any point, including for validation.
Censored data	Observations the system structurally never produces, because the gate it would inform prevents them.
Control group	Answers regenerated despite passing the gate, purely to produce censored observations.
Position / slot	Which of the two prompt positions a candidate occupied for a given question.
Identity	Which physical model produced a candidate, independent of its position.
Lean	The signed difference in similarity between the final answer and each candidate.
Discrimination	An evaluator's capacity to score good and bad answers differently.
Blind	An evaluator sees neither model identity nor the other evaluator's scores.
Census row	A record regenerated because it genuinely failed the gate; sampling weight 1.0.
Sensitivity	Of improvable answers, the weighted fraction correctly flagged for regeneration.
Specificity	Of satisfactory answers, the weighted fraction correctly left alone.

Table K.1: Terms as used in this report

Two of these warrant emphasis because misreading them would misrepresent the system's claims. The confidence score is a ranked signal, not a probability: a score of 0.80 means better supported than 0.60, not 80 per cent likely to be correct. And "calibrated" throughout this report refers to the choice of an operating point on that ranking, in the sense used in diagnostic testing, not to agreement between stated confidence and

empirical accuracy - which, as Section 2.9 explains, cannot be established without the ground truth this methodology deliberately does without.

Appendix L: Development Chronology

The project kept a running engineering log, and its 38 entries are the primary source for Chapter 5. The sequence is summarised here because the order in which things were discovered explains several decisions that look arbitrary in isolation.

Entries 1-6 cover the initial build: the eight-factor dual-evaluator design, the weighted confidence score, Wikidata integration alongside Wikipedia, the first full dataset regeneration, and the first round of infrastructure failures.

Entries 7-16 are the judge-selection sequence described in Section 5.4. They record the positivity-bias discovery, the broader model search, the discrimination test that corrected the methodology, the switch to CompassJudger-1, the anchoring bug that made the two evaluators non-independent, two rubric revisions, the discovery that CompassJudger-1 was flat-scoring on 57.5% of evaluations, and the final adoption of qwen3.5:9b with reasoning disabled.

Entries 17-19 cover the position and identity bias fix, a checkpoint analysis at 670 of 5,000 questions, and the move of process supervision out of the interactive session into a scheduled task.

Entries 20-28 are the threshold work: the decision to use label-free objectives in the absence of ground truth, the introduction of the control group, three successive threshold reads of increasing rigour, the ROC and bootstrap analysis, and two bugs found along the way - the uncalibrated retrieval gate and a stale threshold constant in the results viewer.

Entries 29-34 cover the model-bias investigation: whether the pipeline favoured one generator, the viewer bug that made the data look far more lopsided than it was, the verifier discrimination test, raising the control-sample rate to finish the recheck, dataset completion at 5,000, and the final threshold recalculation with its sampling-weight correction.

Entries 35-38 are the closing sequence: deliverables, the threshold change from 0.75 to 0.72, the combiner bias decomposition, and the controlled replay that tested the fix and contradicted the reasoning behind it.

Two patterns run through the log and are worth stating as process findings rather than results. First, most of the errors in Table 5.4 were found because a number looked too clean, not because anything failed; a pipeline that never crashes can still be wrong throughout. Second, several fixes revealed worse problems than the ones they solved - the independence fix exposed the Judge's flat-scoring, and the position-randomisation fix left a display path behind that produced a false conclusion months later. A fix is not finished when the code changes; it is finished when every consumer of the changed behaviour has been checked.

Appendix M: Reproducing the Results

The pipeline is started by running the model runtime, then the web service, and opening the local address it prints. The batch harness generates the corpus and is restartable.

The analyses reported in Chapter 5 are each a single script. training/optimal_threshold_roc.py performs the threshold calibration and prints the ROC table, the AUC, the Youden optimum and the bootstrap interval. training/combiner_bias_analysis.py performs the bias decomposition over the full corpus and writes a per-record cache so that follow-up analysis requires no re-encoding. training/combiner_replay.py runs the controlled replay and scores it.

deliverables/report/make_figures.py regenerates every figure in Chapter 5 from the live corpus, and deliverables/report/build_report.js rebuilds this document.

Hardware used: a single workstation with an NVIDIA RTX 4090, 24 GB of video memory. The six models exceed that if co-resident, so the pipeline runs at concurrency one and the runtime evicts weights between stages.